Práctica 4. Node.js

Desarrollo de Sistemas Distribuidos. Grupo DSD-3.

Grado en Ingeniería Informática

Curso 2025-2026.

ETSIIT. Universidad de Granada.





UNIVERSIDAD
DE GRANADA

Autor:
Ángel Rodríguez Faya

Profesor:
José Ángel Segura Muros

Fecha de entrega:
22 de mayo de 2026

Índice

1. Introducción. **2**

2. Parte 1.- Implementación de ejemplos. **3**
2.1. Ejemplo 1.- Hola Mundo. 3
2.2. Ejemplo 2.- Calculadora REST. 5
2.3. Ejemplo 3.- Calculadora WEB. 6
2.4. Ejemplo 4.- Socket.io. 8
2.5. Ejemplo 5.- MongoDB. 10

3. Parte 2.- Sistema Domótico. **13**
3.1. Introducción 13
3.2. Arquitectura del Sistema 14
3.2.1. El Servidor Central (`registro_sensores.js`) 14
3.2.2. Nodos Sensores y Emulación (`sensor.html`) 25
3.2.3. Panel de Control de Usuario (`dashboard.html`) 32

4. Diagramas de Secuencia **40**
4.1. Escenario 1: Registro de un Sensor y Telemetría 41
4.2. Escenario 2: Evaluación y Envío de un Aviso (Alerta Crítica) . 41
4.3. Escenario 3: Envío de Videovigilancia (Bypass Multimedia) . . 42

5. Guía de Uso y Ejemplos de Ejecución **43**
5.1. Puesta en Marcha del Sistema 43
5.2. Uso de las Interfaces de Usuario 44
5.3. Ejemplos de Ejecución y Casos de Prueba 47
5.3.1. Caso de Prueba 1: Alerta Crítica por Fuga de Gas en la Cocina 47
5.3.2. Caso de Prueba 2: Regla Compleja Cruzada (Luminosidad y Movimiento) 52
5.3.3. Caso de Prueba 3: Seguridad en el Portal (Videovigilancia Activa) 57

1. Introducción.

Este documento constituye la memoria de la Práctica 4 de la asignatura *Desarrollo de Sistemas Distribuidos*. El objetivo central de este proyecto radica en el diseño, implementación y evaluación experimental de un sistema domótico distribuido, cooperativo y en tiempo real, haciendo uso de tres pilares tecnológicos esenciales:

- **Node.js:** Es una plataforma de ejecución de JavaScript del lado del servidor basada en el motor V8 de Google. Destaca por su modelo de operaciones de I/O no bloqueantes y su arquitectura orientada a eventos (*event-driven*), controlada por un único hilo de ejecución centralizado (*Event Loop*). Esto permite mitigar la sobrecarga asociada a la creación de hilos tradicionales por cada conexión entrante.
- **Socket.io:** Es una biblioteca especializada en habilitar una capa de comunicación bidireccional, síncrona y de alta velocidad entre clientes y servidores sobre el protocolo WebSocket. Su modelo basado en eventos encaja de forma nativa con el patrón de diseño Publicación/Suscripción, idóneo para la propagación inmediata de estados concurrentes.
- **MongoDB:** Es un sistema de base de datos `NoSQL` orientado a documentos que almacena la información en esquemas flexibles de tipo `BSON/JSON`. Proporciona un mecanismo de persistencia ideal para el flujo heterogéneo de telemetría y logs de eventos generados de forma asíncrona en entornos distribuidos.

El propósito último de este proyecto/práctica es el despliegue de una infraestructura inteligente donde múltiples procesos independientes — como nodos sensores, nodos actuadores, un agente lógico de control supervisado y paneles web de usuario (o *dashboards*)— cooperen de manera asíncrona y transparente para garantizar la habitabilidad y eficiencia del entorno que está siendo monitorizado.

2. Parte 1.- Implementación de ejemplos.

Antes de abordar el diseño de la arquitectura completa del sistema domótico, es fundamental que asimilemos los fundamentos de red y los patrones de concurrencia asíncrona que manejan el entorno de Node.js. Por este motivo, la primera fase o parte de la práctica se centra en la implementación, despliegue y análisis de una serie de servicios y ejemplos base propuestos en el guion.

A continuación, vamos a ir viendo cada uno de los ejemplos. Para ello, analizaremos el código fuente de cada uno de los ejemplos, verificar su correcto funcionamiento mediante capturas de pantalla, y la correspondiente justificación técnica de los fenómenos observados.

2.1. Ejemplo 1.- Hola Mundo.

Para poner en marcha este primer ejemplo, abrimos la terminal en el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_1_Hola_Mundo`) y ejecutamos el comando:

```
$ node helloworld.js
```

Capturas de funcionamiento

A continuación se adjuntan las capturas de pantalla que verifican el correcto despliegue del servidor y su respuesta:



Figura 1: Visualización en el navegador de la respuesta enviada por el servidor.

```
angelrf@angel-ubuntu:~/Practica_4_Node_js/codigo/Parte_1_Implementacion_de_Ejemp
los/Ejemplo_1_Hola_Mundo$ node helloworld.js
Servicio HTTP iniciado
{
  host: 'localhost:8080',
  'user-agent': 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/2010010
1 Firefox/150.0',
  accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
  'accept-language': 'es-ES,es;q=0.9,en-US;q=0.8,en;q=0.7',
  'accept-encoding': 'gzip, deflate, br, zstd',
  connection: 'keep-alive',
  cookie: 'phpMyAdmin=6fc530396eb76ccbaff352cbac7b55b4; PHPSESSID=da9e2736ada057
f6910b6463a4dcf300',
  'upgrade-insecure-requests': '1',
  'sec-fetch-dest': 'document',
  'sec-fetch-mode': 'navigate',
  'sec-fetch-site': 'none',
  'sec-fetch-user': '?1',
  priority: 'u=0, i'
}
```

Figura 2: Salida por terminal del servidor mostrando el inicio y las cabeceras recibidas.

Vemos que el script utiliza el módulo nativo `http` de Node.js para levantar un servidor web básico que se queda escuchando peticiones en el puerto `8080`. Cada vez que un cliente (como el navegador) accede, el servidor imprime en la consola las cabeceras de la petición y devuelve un mensaje de texto plano con el contenido “Hola mundo”.

Lo más relevante a destacar de este ejemplo es:

- **La petición no bloqueante (Asíncrona):** El mensaje `'Servicio HTTP iniciado'` se muestra en la terminal de forma inmediata al lanzar el script. Esto demuestra que el método `.listen(8080)` es asíncrono y no detiene la ejecución del programa a esperar conexiones.
- **Uso de Callbacks:** La función interna de `createServer` actúa como un *callback*. No se ejecuta al arrancar el programa, sino que el *Event Loop* (bucle de eventos) la invoca únicamente cuando entra una petición HTTP, gestionando así la concurrencia de forma eficiente en un solo hilo o hebar.

2.2. Ejemplo 2.- Calculadora REST.

Para este segundo ejemplo, lo podemos poner en marcha abriendo en la terminal el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_2_Calculadora_REST`) y ejecutamos el comando:

```
$ node calculadora.js
```

Capturas de funcionamiento

A continuación se adjuntan las capturas de pantalla:

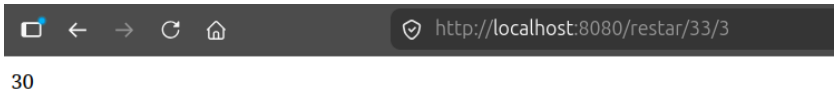


Figura 3: Visualización en el navegador del resultado de restar 33-3

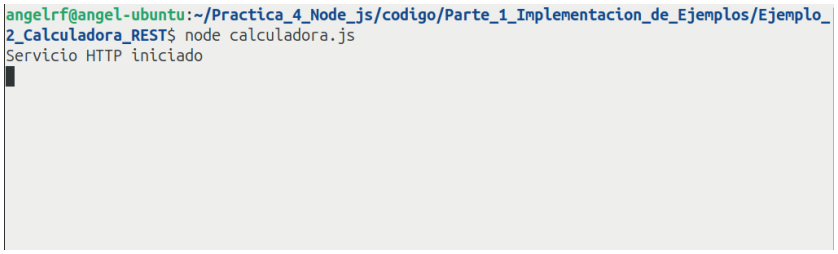


Figura 4: Salida por terminal del servidor mostrando el inicio.

Viendo el código fuente y la figura 3 podemos sacar varias conclusiones/apuntes sobre Node.js:

- El servidor tiene que **procesar la URL** como una cadena de texto, para descomponer la ruta de la petición en un array de parámetros.
- Como todo lo que viaja por HTTP es texto, necesitamos **deserializar los datos** mediante `parseFloat()` para operar con ellos numéricamente y, posteriormente, convertirlos de nuevo a texto (`result.toString()`) para poder transmitirlos en la respuesta HTTP.
- Se realiza un **control de errores**. Se comprueba que el array de parámetros extraído de la URL tenga >= 3 parámetros para poder realizar la operación. En caso de que tenga menos, el sistema no se cae (no hace *crash*), si no que responde con un mensaje de error controlado manteniendo así la **disponibilidad del servicio**

Este ejemplo es una calculadora **REST**. Esto quiere decir que el servidor actúa como una **API**. Solo espera recibir comandos en la URL (ej. /sumar/5/3) y devuelve datos puros (el resultado en texto plano). No tiene diseño, ni botones, ni colores. Está pensado para comunicación máquina a máquina (por ejemplo, para que otro programa o un comando de terminal interactúe con él), tal y cómo hemos visto en las capturas de pantalla.

2.3. Ejemplo 3.- Calculadora WEB.

En este ejemplo, vemos otra calculadora pero a diferencia de la anterior, esta es una calculadora WEB. Es decir, el servidor actúa como un **Servidor Web tradicional y además como API**. Está orientado a humanos y no a máquinas como el ejemplo anterior. Cuando entras a la ruta base (/), el servidor lee del disco duro y nos devuelve una página web completa (`calc.html`). Esa página web tiene una interfaz visual (un formulario,

un desplegable) y contiene código JavaScript que, de forma invisible (usando peticiones `fetch`), hace las llamadas REST al servidor para obtener el resultado sin tener que recargar la página.

Para poder ejecutar este ejemplo, abrimos la terminal y nos situamos en el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_3_Calculadora_WEB`) y ejecutamos el comando:

```
$ node calculadora-web.js
```

Capturas de funcionamiento

A continuación se adjuntan las capturas de pantalla:

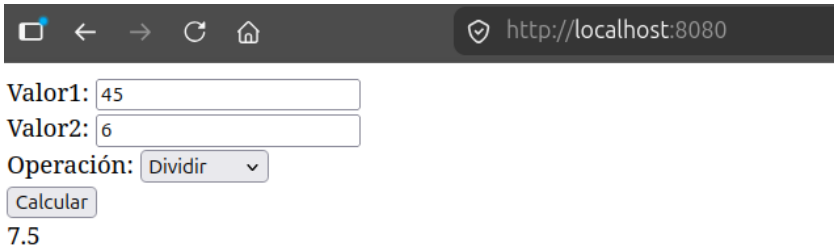


Figura 5: Visualización en el navegador de la calculadora comportándose como una página web normal. Divide 45/6.

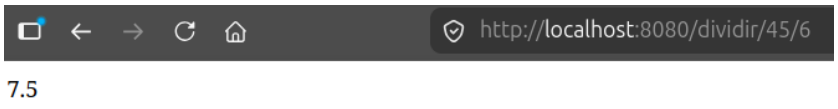


Figura 6: Visualización en el navegador de la calculadora comportándose como una API. Divide 45/6.



Figura 7: Salida por terminal del servidor mostrando el inicio.

Este ejemplo da un paso más hacia una arquitectura web real al separar el *Frontend* (interfaz) del *Backend* (lógica). El servidor ahora realiza dos funciones distintas dependiendo de la ruta (*routing*) solicitada:

- En la figura 5 podemos ver el **servicio de archivos estáticos (I/O Asíncrono)**. Si el cliente accede a la raíz (`/`), el servidor utiliza los módulos nativos `path` y `fs` (File System) para buscar el archivo `calc.html` en el disco duro. Emplea la función asíncrona `readFile` junto con un *callback* `(err, data)` para leer el archivo sin bloquear el hilo principal, y lo devuelve al cliente con la cabecera `text/html`. Comportándose como una página web real.

- En cambio, en la figura 6, vemos en acción el **consumo de API mediante AJAX (`fetch`).** En el lado del cliente (el navegador), el archivo HTML contiene un script que intercepta el envío del formulario (`submit`). En lugar de recargar la página, utiliza la API `fetch` de JavaScript (basada en Promesas con `.then()`) para construir la URL dinámica (ej. `/sumar/5/3`), solicitar el resultado al servidor en segundo plano y actualizar únicamente la etiqueta `<span>` del resultado en el DOM. Este comportamiento es parecido al de la calculadora del ejemplo anterior.

2.4. Ejemplo 4.- Socket.io.

Ahora ya estamos ante una página web **en tiempo real y bidireccional**. En este ejemplo voy a comentar primero las anotaciones o cueriosidades y luego vemos los ejemplos.

- Ya nos olvidamos de las peticiones, pues ahora trabajaremos con **eventos**. Esto ocurre porque, a diferencia de HTTP que obliga a que el navegador siempre pregunte primero para que el servidor responda, Socket.io (basado en WebSockets) mantiene un "tubo"de comunicación siempre abierto. Ahora, el servidor puede enviar datos al navegador por iniciativa propia sin que nadie se lo pida.

- Estamos ante el **Patrón de Suscripción/Emisión** (`emit` y `on`), que se diferencia con el **Patrón *Observer*** en que los publicadores (emisores) y los suscriptores (receptores) no se comunican directamente entre sí. Utilizan un intermediario (*Message Broker* o Bus de Eventos). Los publicadores envían mensajes a un çanal.º "temaz los suscriptores escuchan ese canal. Es por eso que la comunicación ya no se basa en URLs (rutas), sino en .ev"entosçon nombres inventados por el programador.
  - Usamos `.emit('nombre-evento', datos)` para disparar un mensaje por el "tubo".
  - Usamos `.on('nombre-evento', callback)` para quedarnos escuchando.
  - Por ejemplo, el navegador emite `output-evt` con el texto "¡Hola Servicio!", y el servidor lo escucha para contestar.
- Se realiza una **Difusión o *Broadcast*** (`io.sockets.emit`): El servidor de Node.js guarda un array llamado `allClients` con las direcciones IP y puertos de todos los conectados. Cuando alguien nuevo entra (o cuando alguien cierra la pestaña y se dispara el evento `disconnect`), el servidor usa `io.sockets.emit('all-connections', allClients)`, para mantenerla lista actualizada a todos los clientes que están conectados a la vez.
- El **DOM es Dinámico** (Frontend): En el archivo HTML, encontramos código JavaScript que está "suscrito.al evento `all-connections`. Cada vez que el servidor lanza esa nueva lista, el cliente borra la lista antigua de la pantalla y dibuja la nueva al instante, sin tener que darle al botón de actualizar del navegador.

Para poder ejecutar este ejemplo, abrimos la terminal y nos situamos en el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_4_Socketio`) y ejecutamos el comando:

```
$ node connections.js
```

Capturas de funcionamiento

A continuación se adjuntan las capturas de pantalla:

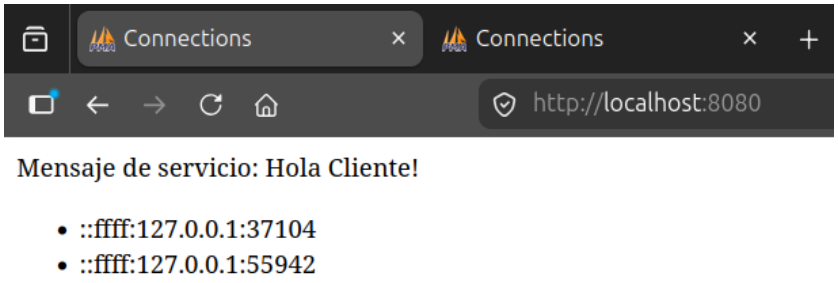


Figura 8: Visualización en el navegador de la dirección Ip y el puerto de los dos clientes conectados a nuestro servidor.

```
angelrf@angel-ubuntu:~/Practica_4_Node_js/codigo/Parte_1_Implementacion_de_Ejemplos/Ejemplo_
4_Socketio$ node connections.js
Servicio Socket.io iniciado
Nueva conexión de ::ffff:127.0.0.1:37104
Nueva conexión de ::ffff:127.0.0.1:55942
```

Figura 9: Salida por terminal del servidor mostrando el inicio.

Como podemos observar en las figuras, el cliente web muestra los clientes conectados. La dirección IP es la misma puesto que he abierto dos pestañas en mi misma máquina, eso sí, el puerto obviamente si que cambia

2.5. Ejemplo 5.- MongoDB.

En este ejemplo vamos un paso más allá: incorporar una base de datos real. Utilizaremos `MongoDB`, que es un sistema de gestión de bases de datos de tipo NoSQL . Este SGBD utiliza colecciones de entradas en formato JSON, en lugar de tablas que encontramos en SQL. Cada entrada puede tener un conjunto de claves y valores arbitrarios. Para obtener información, realizamos consultas similares a las utilizadas en SQL.

En este caso se utiliza para almacenar los clientes que se han conectado en nuestro servidor. En el ejemplo 4, los clientes conectados se almacenaban en un array de JS. Tenía como inconveniente que los datos eran volátiles, pues cuando apagamos el servidor, el array obviamente se borraba. En este ejemplo, podemos ver en las capturas de pantalla que las direcciones de los clientes se almacenan en la BD, porque se almacenan junto a la fecha y hora en la que se conectaron, y vemos que hay dos fechas distintas: 24 de abril y 21 de mayo.

Para poder ejecutar este ejemplo, abrimos la terminal y nos situamos en el directorio del archivo (que es `Parte_1_Implementacion_de_Ejemplos/Ejemplo_5_MongoDB`) y ejecutamos los comandos:

- Para arrancar el servicio de MongoDB (previamente debe estar instalado en nuestro PC)

```
$ sudo systemctl start mongod
```

- Para lanzar el servidor:

```
$ node mongo-test.js
```

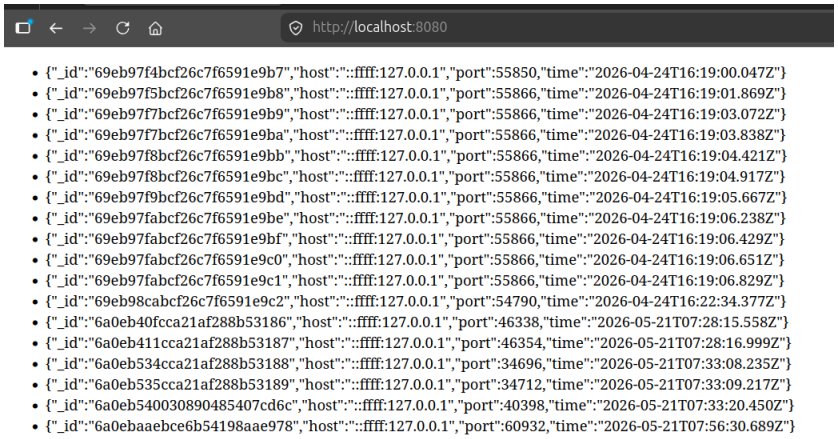


Figura 10: Visualización en el navegador de os clientes que se han conectado alguna vez al servidor.



Figura 11: Salida por terminal del servidor mostrando el inicio.

Observando las figuras 10 y 11, vemos que, a diferencia del Ejemplo 4 (donde solo imprimíamos texto plano formateado con la IP y el puerto), aquí vemos que los datos aparecen en pantalla como objetos JSON puros encadenados. El campo llamado `_id` con un código alfanumérico largo (`ObjectId`). Este elemento no lo ha programado el cliente; lo genera e inyecta MongoDB automáticamente para que actúe como clave primaria única del documento dentro de la colección.

Además, se utilizan Promesas o *Promises* para un *Flujo Asíncrono* , evitando bloquear el único hilo de ejecución de Node.js durante las operaciones de Entrada/Salida (I/O) con el disco. El driver de MongoDB utiliza las *Promises*. Toda la lógica de red se anida secuencialmente dentro de métodos `.then()`, asegurando que el servidor pueda seguir escuchando nuevos sockets mientras procesa consultas concurrentes.

En el Ejemplo 4, el servidor tomaba un rol activo enviando la lista de conexiones por iniciativa propia nada más establecerse el enlace. En el Ejemplo 5, el flujo lo conduce el cliente web de forma reactiva:

1. El servidor le dice al cliente cuál es su dirección (`my-address`).
2. El cliente añade la fecha actual y ordena activamente guardarla (`poner`).
3. Finalmente, el cliente solicita expresamente recuperar el histórico (`obtener`).

Por último, destacar que el Servidor no tiene estado (*Stateless*). Es lo que habíamos comentado al principio del ejemplo, dado que el servidor de Node.js ya no mantiene ninguna lista viva de sockets en su memoria volátil, se elimina por completo la necesidad de escuchar el evento `disconnect` en `mongo-test.js`. Al servidor le resulta indiferente si el usuario cierra el navegador, ya que el registro queda guardado de manera permanente e inmutable en MongoDB.

3. Parte 2.- Sistema Domótico.

3.1. Introducción

El objetivo de esta segunda parte de la práctica radica en el diseño, implementación y evaluación experimental de un sistema domótico distribuido, cooperativo y en tiempo real, haciendo uso de los pilares tecnológicos que ya comentamos en la sección anterior: **Node.js**, **Socket.io** y **MongoDB**.

El motivo de realizar esta parte sobre un sistema domótico es la de simular el despliegue de una infraestructura inteligente donde múltiples procesos independientes cooperen de manera asíncrona y transparente. Para ello, el sistema se ha diseñado cumpliendo los siguientes requisitos y características arquitectónicas:

1. **Emulación de Nodos Sensores y Actuadores:** Creación de clientes web independientes que actúan como sensores físicos (Temperatura, Luminosidad, Gas, Movimiento y Videovigilancia), publicando telemetría tanto de forma periódica (5 segundos), como de forma manual; hacia el servidor central.

2. **Agente Lógico Centralizado:** Implementación de un motor de reglas en el servidor que evalúa la telemetría entrante en tiempo real para tomar decisiones sobre los actuadores (ej. encender el aire acondicionado, bajar persianas o activar alarmas de gas). Se incluyen tanto reglas simples puramente reactivas, como **reglas complejas con memoria** (ej. mantener las luces encendidas un tiempo determinado tras dejar de detectar movimiento).

3. **Patrón de "Gemelo Digital":** El servidor mantiene una representación en memoria (*Digital Twin*) del estado físico real de los actuadores por zonas de la casa, asegurando la consistencia de los datos y evitando "fantasmas" en la interfaz web‘. Un **Gemelo Digital** (*Digital Twin*) es una réplica virtual de un objeto, sistema o proceso físico del mundo real. No es solo un dibujo o un modelo estático en 3D; **es una representación informática dinámica que se actualiza en tiempo real utilizando los datos que envían los sensores del mundo físico.**

4. **Panel de Control (Dashboard):** Una interfaz web central, separada de la lógica de negocio y diseñada para el **usuario** , que se actualiza reactivamente mediante *broadcast* mostrando la lectura de los sensores, el estado de los actuadores y un registro de alertas críticas, así

como un panel que muestra en tiempo real el **Circuito Cerrado de Videovigilancia**.

5. **Telegram Bot API.** Se ha desarrollado una Plataforma de integración a través de peticiones HTTP REST que permite expandir los límites físicos del sistema domótico, enviando alertas críticas (al igual que en el *Dashboard*) y contenido multimedia del Circuito de Videovigilancia directamente al dispositivo del usuario de forma instantánea.

Con estas implementaciones añadidas (además de la implementación básica), se cubren todos los requisitos que se piden en el guion de prácticas. A continuación, se detalla la arquitectura del sistema y su modelo de funcionamiento.

3.2. Arquitectura del Sistema

El sistema domótico se ha diseñado siguiendo un patrón de arquitectura *Cliente-Servidor Distribuido* y *Orientado a Eventos*. La comunicación principal se basa sobre conexiones persistentes de baja latencia mediante el protocolo WebSocket (gestionado a través de *Socket.io*), rompiendo con el esquema tradicional de petición/respuesta síncrona de HTTP.

A continuación, se desglosan los componentes del sistema divididos de forma lógica entre el núcleo central (Servidor) y los procesos periféricos (Clientes).

3.2.1. El Servidor Central (`registro_sensores.js`)

El servidor central actúa como el cerebro de toda la casa inteligente (es el **Bus de Evnetos** y el **Agente Lógico** que toma las decisiones). Sus tareas principales dentro de nuestro sistema son las siguientes:

- **Atender a los clientes:** Escucha en el puerto `8080` y mantiene abiertas las conexiones en tiempo real (mediante WebSockets) con todos los dispositivos. Además, es capaz de identificar si el que se ha conectado es un sensor enviando datos o es el panel de control del usuario.
- **Mantener el "Gemelo Digital":** Para evitar fallos o desincronizaciones, el servidor guarda en su propia memoria una copia exacta (un reflejo) del estado actual de todos los actuadores de la casa (luces, persianas, etc.). Cualquier cambio pasa primero por aquí.

- **Tomar decisiones (Agente Lógico):** Analiza al vuelo cada dato que le llega de los sensores. Si detecta algo fuera de lo normal (por ejemplo, que el nivel de gas supera los 300 o que hay movimiento en el portal), ejecuta automáticamente las reglas que hemos programado para lanzar alertas o cambiar el estado de los actuadores.

- **Guardar el historial:** Utiliza la base de datos de `MongoDB` para ir anotando todo lo que ocurre. Lo hace de forma asíncrona (usando Promesas), lo que significa que el servidor puede seguir guardando registros sin quedarse çongelado"ni desatender al resto de la casa.

  Su código es el siguiente:

```
const http = require('node:http');
const {join} = require('node:path');
const {readFile} = require('node:fs');
const {MongoClient} = require('mongodb');
const {Server} = require('socket.io');

const TelegramBot = require('node-telegram-bot-api');

// =============================================================================
//                  Configuración y variables globales.
// =============================================================================

// ------- Configuración básica: puerto y conexión a MongoDB. -------
const PORT = process.env.PORT || 8080;
const MONGODB_URI = process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017';
const MONGODB_DB_NAME = process.env.MONGODB_DB_NAME || 'domotica';

// ------- Objetos "freeze" para el estado de los actuadores -------

const EstadoAC = Object.freeze({
    APAGADO: 'APAGADO',
    ENCENDIDO: 'ENCENDIDO'
});

const EstadoPersiana = Object.freeze({
    ABIERTA: 'ABIERTA',
    CERRADA: 'CERRADA'
});

const EstadoAlarmaGas = Object.freeze({
    INACTIVA: 'INACTIVA',
    ACTIVA: 'ACTIVA'
});

const EstadoLuz = Object.freeze({
    APAGADA: 'APAGADA',
    ENCENDIDA: 'ENCENDIDA'
});

// ------- Variables globales compartidas para reutilizar la conexion y la coleccion a la base de datos. -------
let mongoClient;        // Cliente de MongoDB
let sensoresCollection; // Colección de MongoDB donde se almacenarán los sensores registrados.
let historialCollection; // Colección de MongoDB donde se almacenarán las mediciones de los sensores.
```

```
44
45  let estadoActuadores = {  // Estado actual de los actuadores, compartido para que el agente de control y el dashboard
46      aireAcondicionado: {
47          "Salon": EstadoAC.APAGADO,
48          "Dormitorio 1": EstadoAC.APAGADO,
49          "Cocina": EstadoAC.APAGADO,
50      },
51      persiana: {
52          "Salon": EstadoPersiana.ABIERTA,
53          "Dormitorio 1": EstadoPersiana.ABIERTA,
54          "Dormitorio 2": EstadoPersiana.ABIERTA,
55          "Cocina": EstadoPersiana.ABIERTA,
56          "Bano": EstadoPersiana.ABIERTA
57      },
58      alarmaGas: {
59          "Cocina": EstadoAlarmaGas.INACTIVA,
60      },
61      luz: {
62          "Salon": EstadoLuz.APAGADA,
63          "Dormitorio 1": EstadoLuz.APAGADA,
64          "Dormitorio 2": EstadoLuz.APAGADA,
65          "Cocina": EstadoLuz.APAGADA,
66          "Bano": EstadoLuz.APAGADA
67      }
68  };
69
70  // Memoria para eventos complejos: Guardamos la última luminosidad conocida de cada zona
71  let memoriaLuminosidad = {};
72
73  // ------- Constantes/Umbrales para el agente de control -------
74  const MAX_TEMP = 30.0;  // grados Celsius
75  const MIN_TEMP = 17.0; // grados Celsius
76  const MAX_LUMINOSIDAD = 700; // Lux
77  const MAX_PPM_GAS = 300; // Partículas por millón (ppm)
78
79  // ------- Configuración del bot de Telegram -------
80  const TELEGRAM_TOKEN = '8702302349:AAHUwKTa4EM73Qedbv9qeDpPYKl3dvwu2sc';
81  const telegramBot = new TelegramBot(TELEGRAM_TOKEN, {polling: true});
82
83  // Para comprobar si hay errores en la conexión con Telegram
84  telegramBot.on('polling_error', (error) => {
85      console.log('[ERROR TELEGRAM]', error.message);
86  });
87
88  let miChatId = null; // Variable para almacenar el chat ID del usuario que interactúa con el bot
89
90  // Cuando el cliente hable por primera vez con el bot, guardamos su chat ID para enviarle alertas posteriormente.
91  telegramBot.onText(/\/start/, (msg) => {
92      miChatId = msg.chat.id;
93      telegramBot.sendMessage(miChatId, '¡Hola! Soy tu asistente de domótica. Te enviaré alertas importantes sobre el e
94      console.log(`[TELEGRAM] Usuario registrado para alertas: ${miChatId}`);
95  });
96
97  // ============================================================================
98  //                    Funciones para manejar MongoDB.
99  // ============================================================================
100
101 // Abre la conexion con MongoDB y guarda la coleccion a utilizar.
102 async function connectToMongo() {
103         //Inicializamos el cliente de la base de datos
104     mongoClient = new MongoClient(MONGODB_URI);
105
```

```
106         //Esperamos a que conecte
107     await mongoClient.connect();
108 
109         //Obtenemos la base de datos y creamos/cargamos las colecciones necesarias
110     const db = mongoClient.db(MONGODB_DB_NAME);
111     sensoresCollection = db.collection('sensores');
112     historialCollection = db.collection('historial');
113 }
114 
115 
116 // Inserta un sensor nuevo en MongoDB y devuelve el documento insertado.
117 async function registerSensor(sensorData) {
118         //Creamos la entrada de la tabla a partir de los datos pasados en el parametro sensorData
119     const sensor = {
120         name: sensorData.name,
121         location: sensorData.location,
122         type: sensorData.type,
123         registeredAt: new Date()
124     };
125 
126         //Insertamos la entrada en la coleccion "sensores"
127     await sensoresCollection.insertOne(sensor);
128     return sensor;
129 }
130 
131 
132 // Muestra por consola todos los sensores registrados.
133 async function logRegisteredSensors() {
134         //Obtenemos todas las entradas de la coleccion "sensores"
135     const sensors = await sensoresCollection
136         .find({}, {projection: {_id: 0}})
137         .sort({registeredAt: -1})
138         .toArray();
139 
140         //Las mostramos por consola en formato tabla
141     console.log('Sensores registrados actualmente en la base de datos:');
142     console.table(sensors);
143 }
144 
145 // ============================================================================
146 //                       SERVIDOR HTTP (Servir archivos)
147 // ============================================================================
148 
149 // Creamos el servidor http.
150 const httpServer = http.createServer((request, response) => {
151     const {method, url} = request;
152     let filePath = null;
153 
154         //Para el home y la pagina sensores, cargar sensor.html, para la pagina dashboard, cargar dashboard.html
155     if(method === 'GET' && (url === '/' || url === '/sensor.html')) {
156         filePath = join(process.cwd(), 'sensor.html');
157 
158     } else if( method === 'GET' && url === '/dashboard.html') {
159         filePath = join(process.cwd(), 'dashboard.html');
160     }
161 
162     if(filePath) {
163         //Leemos el contenido de sensor.html y lo guardamos en data
164         readFile(filePath, (err, data) => {
165             if(err) {
166                 response.writeHead(500, {'Content-Type': 'text/plain; charset=utf-8'});
167                 response.end('No se pudo cargar el HTML.');
```

```
168             } else {
169                 //Devolvemos al navegador una página creada a partir de data
170                 response.writeHead(200, {'Content-Type': 'text/html; charset=utf-8'});
171                 response.end(data);
172             }
173             response.end();
174         });
175     } else {
176         //En caso de que la ruta no sea ni '/' ni '/dashboard.html', devolvemos un error 404
177         response.writeHead(404, {'Content-Type': 'text/plain; charset=utf-8'});
178         response.end('Ruta no encontrada.');
179         response.end();
180     }
181 });
182 
183 // ================================================================================
184 //                   SERVIDOR EN TIEMPO REAL (Socket.IO)
185 // ================================================================================
186 
187 // Iniciamos el servidor HTTP y Socket.IO.
188 const io = new Server(httpServer); // Engancha Socket.io al servidor HTTP, abriendo el canal para WebSockets
189 
190 // -------- Inicializamos la infraestructura del servidor ---------
191 async function startServer() {
192     try {
193                 //Levantamos mongo
194         await connectToMongo();
195 
196                 // Levantamos el servidor web en el puerto
197         httpServer.listen(PORT, () => {
198             console.log(`Servidor basico escuchando en http://localhost:${PORT}`);
199         });
200     } catch(err) {
201         console.error('No se pudo iniciar el servidor:', err.message);
202         process.exit(1);
203     }
204 }
205 
206 // Manejar la conexion de Socket.IO
207 io.on('connection', (socket) => {
208     console.log('Nuevo nodo conectado: ' + socket.id);
209 
210     // Al conectarse el Dashboard, le enviamos el estado actual de los actuadores para que se muestre correctamente en
211     socket.emit('dashboard:update-actuadores', estadoActuadores);
212 
213     // [Evento 1]: Registro de sensores
214     socket.on('sensor:register', async (payload) => { // payload son los datos del formulario enviados por el cliente
215         const sensorData = {
216             //Extraemos los datos del payload, asegurándonos de que sean strings y eliminando espacios innecesarios.
217             name: String(payload?.name || '').trim(),
218             location: String(payload?.location || '').trim(),
219             type: String(payload?.type || '').trim()
220         };
221 
222         // Validamos que se hayan proporcionado los campos obligatorios.
223         if(!sensorData.name || !sensorData.location || !sensorData.type) {
224             // Emitimos un evento de error al cliente que intenta registrar el sensor.
225                     socket.emit('sensor:error', 'Faltan campos obligatorios.');
226                 } else {
227                     try {
228                         //Intentamos incluir el sensor en la BD
229                         const sensor = await registerSensor(sensorData);
```

```
230
231                             //Mostramos la colección de sensores completa
232                             await logRegisteredSensors();
233                             socket.emit('sensor:registered', sensor);
234
235                         } catch(err) {
236                             //En caso de error, emitir evento de error al sensor.
237                             socket.emit('sensor:error', 'Error registrando el sensor en la base de datos.');
238                         }
239                 }
240     });
241
242     // [Evento 2]: Recepción de medidas del sensor
243     socket.on('sensor:data', async (payload) => {
244         try {
245             // Creamos el documento
246             const medidaNueva = {
247                 sensorId: payload.sensorId,
248                 name: payload.name,
249                 type: payload.type,
250                 location: payload.location,
251                 value: parseFloat(payload.value),
252                 timestamp: new Date()
253             };
254
255             // 1. Insertamos el documento en la colección "historial"
256             await historialCollection.insertOne(medidaNueva);
257             console.log('[BD] Nueva medida registrada:', medidaNueva);
258
259             // 2. Reenviamos (en tiempo real) la medida a todos los clientes conectados (incluido el que la envió) par
260             io.sockets.emit('dashboard:update-sensor', medidaNueva);
261
262             // 3. Llamamos al agente
263             ejecutarAgente(medidaNueva);
264
265         } catch(err) {
266             console.error('[ERROR] Error registrando la medida en la base de datos:', err.message);
267         }
268     });
269
270     // ================================================================================
271     //                     CONTROL MANUAL DESDE EL DASHBOARD
272     // ================================================================================
273     // [Evento 3]: Control manual del AC desde el dashboard
274     socket.on('ui:toggle-ac', (ubicacion) => {
275         if(!ubicacion)
276             return; // Si no se especifica una ubicación, no hacemos nada
277
278         estadoActuadores.aireAcondicionado[ubicacion] =
279             (estadoActuadores.aireAcondicionado[ubicacion] === EstadoAC.APAGADO) ? EstadoAC.ENCENDIDO : EstadoAC.APAG
280
281         console.log(`[MANUAL] A/C de ${ubicacion} cambiado a: ${estadoActuadores.aireAcondicionado[ubicacion]}`);
282         io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
283     });
284
285     // [Evento 4]: Control manual de la persiana desde el dashboard
286     socket.on('ui:toggle-blind', (ubicacion) => {
287         if(!ubicacion)
288             return; // Si no se especifica una ubicación, no hacemos nada
289
290         estadoActuadores.persiana[ubicacion] =
291             (estadoActuadores.persiana[ubicacion] === EstadoPersiana.ABIERTA) ? EstadoPersiana.CERRADA : EstadoPersia
```

```
292
293         console.log(`[MANUAL] Persiana de ${ubicacion} cambiada a: ${estadoActuadores.persiana[ubicacion]}`);
294         io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
295     });
296
297     // [Evento 5]: Control manual de la alarma de gas desde el dashboard
298     socket.on('ui:toggle-gas-alarm', (ubicacion) => {
299         if(!ubicacion)
300             return; // Si no se especifica una ubicación, no hacemos nada
301
302         estadoActuadores.alarmaGas[ubicacion] =
303             (estadoActuadores.alarmaGas[ubicacion] === EstadoAlarmaGas.INACTIVA) ? EstadoAlarmaGas.ACTIVA : EstadoAla
304
305         console.log(`[MANUAL] Alarma de gas de ${ubicacion} cambiada a: ${estadoActuadores.alarmaGas[ubicacion]}`);
306         io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
307     });
308
309     // [Evento 6]: Enviar sensores existentes al abrir la página
310     socket.on('sensor:request-sensors', async () => {
311         try {
312             const listaSensores = await sensoresCollection.find().toArray();
313             // Solamente los enviamos al cliente que lo ha solicitado
314             socket.emit('sensor:response-sensors', listaSensores);
315         } catch(err) {
316             console.error('[ERROR] Error obteniendo la lista de sensores de la base de datos:', err.message);
317         }
318     });
319
320     // [Evento 6]: Control manual de la luz desde el dashboard
321     socket.on('ui:toggle-light', (ubicacion) => {
322         if(!ubicacion)
323             return;
324
325         estadoActuadores.luz[ubicacion] =
326             (estadoActuadores.luz[ubicacion] === EstadoLuz.APAGADA) ? EstadoLuz.ENCENDIDA : EstadoLuz.APAGADA;
327
328         console.log(`[MANUAL] Luz de ${ubicacion} cambiada a: ${estadoActuadores.luz[ubicacion]}`);
329         io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
330     });
331
332     // [Evento X]: Manejar la desconexión del cliente.
333     socket.on('disconnect', () => {
334         console.log('El nodo ' + socket.id + ' se ha desconectado.');
335     });
336 });
337
338 // ============================================================================
339 //                   AGENTE DE CONTROL
340 // ============================================================================
341 function notificarAlerta(mensaje) {
342     // 1. Enviar mensaje de alerta al dashboard
343     io.sockets.emit('dashboard:alert', mensaje);
344
345     // 2. Enviar mensaje de alerta por Telegram (si el usuario se ha registrado)
346     if (miChatId) {
347         telegramBot.sendMessage(miChatId, mensaje, {parse_mode: 'HTML'});
348     } else {
349         console.log('[TELEGRAM] La Alerta ha sido retenida, no se ha inciado el bot con /start');
350     }
351 }
352
353 function ejecutarAgente(medida) {
```

```
354
355         const valor = medida.value;
356         const tipo = medida.type;
357         const ubicacion = medida.location;
358
359         // Reglas de control para el aire acondicionado basadas en la temperatura
360         if(tipo === 'Temperatura') {
361
362             // Regla 1: Si la temperatura es mayor que MAX_TEMP -> Alerta
363             if(valor > MAX_TEMP) {
364                 let msg = ` <strong>¡ALERTA!</strong> Se ha detectado una temperatura (${valor}°C) por encima del máximo
365
366                 // Si tiene actuador (A/C) y está apagado, lo encendemos automáticamente
367                 if(estadoActuadores.aireAcondicionado[ubicacion] === EstadoAC.APAGADO){
368                     estadoActuadores.aireAcondicionado[ubicacion] = EstadoAC.ENCENDIDO;
369                     io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
370                     // Notificamos la alerta tanto en el dashboard como por Telegram
371                     msg = msg + ` <strong>Encendiendo A/C automáticamente.</strong>`;
372                 }
373
374                 // Enviamos la alerta al dashboard y por Telegram
375                 notificarAlerta(msg);
376             }
377
378             // Regla 2: Si la temperatura es menor que MIN_TEMP -> Alerta
379             if(valor < MIN_TEMP) {
380                 let msg = ` <strong>¡ALERTA!</strong> Se ha detectado una temperatura (${valor}°C) por debajo del mínimo
381
382                 // Si tiene actuador (A/C) y está encendido, lo apagamos automáticamente
383                 if(estadoActuadores.aireAcondicionado[ubicacion] === EstadoAC.ENCENDIDO){
384                     estadoActuadores.aireAcondicionado[ubicacion] = EstadoAC.APAGADO;
385                     io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
386                     msg = msg + ` <strong>Apagando A/C automáticamente.</strong>`;
387                 }
388
389                 // Enviamos la alerta al dashboard y por Telegram
390                 notificarAlerta(msg);
391             }
392         }
393
394
395         // Regla de control para la persiana
396         if(tipo === 'Luminosidad') {
397
398             // Guardamos el valor de luminosidad en la memoria para esta ubicación, para poder usarlo en reglas más comple
399             memoriaLuminosidad[ubicacion] = valor;
400
401             // Regla 3: Si la luminosidad es mayor que MAX_LUMINOSIDAD -> Alerta
402             if(valor > MAX_LUMINOSIDAD) {
403                 let msg = ` <strong>¡ALERTA!</strong> Luminosidad máxima superada (${valor} lux) en ${ubicacion}.`;
404
405                 // Si tiene actuador (persiana) y está abierta, la cerramos automáticamente
406                 if(estadoActuadores.persiana[ubicacion] === EstadoPersiana.ABIERTA) {
407                     estadoActuadores.persiana[ubicacion] = EstadoPersiana.CERRADA;
408                     io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
409                     msg = msg + ` <strong>Cerrando persiana automáticamente.</strong>`;
410                 }
411
412                 // Enviamos la alerta al dashboard y por Telegram
413                 notificarAlerta(msg);
414             }
415         }
```

```
416
417         // Regla de control para el gas
418         if(tipo === 'Gas') {
419
420             // Regla 4: Si el nivel de gas es mayor que MAX_PPM_GAS -> Enviar alerta y sonar alarma
421             if(valor > MAX_PPM_GAS) {
422                 let msg = ` <strong>¡ALERTA CRÍTICA!</strong>  Se ha detectado un nivel de gas elevado (${valor} ppm)` +
423                     ` en ${ubicacion}.`;
424
425                 // Si tiene actuador (alarma de gas) y está inactiva, la activamos automáticamente
426                 if(estadoActuadores.alarmaGas[ubicacion] === EstadoAlarmaGas.INACTIVA) {
427                     estadoActuadores.alarmaGas[ubicacion] = EstadoAlarmaGas.ACTIVA;
428                     io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
429                     msg = msg + ` <strong>Se ha activado la alarma de gas. Por favor, revisa tu hogar inmediatamente.</st
430                 }
431
432                 // Enviamos la alerta al dashboard y por Telegram
433                 notificarAlerta(msg);
434             }
435         }
436
437         // Regla de control compleja: Si hay poca luz y se detecta movimiento, encender la luz automáticamente (si existe
438         if(tipo === 'Movimiento') {
439
440             if(valor === 1) { // Si se detecta movimiento
441                 // notificarAlerta(` Movimiento detectado en £{ubicacion}.`);
442
443                 // Comprobamos la luminosidad actual de esa ubicación en la memoria
444                 // Si no tenemos un valor de luminosidad para esa ubicación, asumimos que está iluminada (500 lux)
445                 const luminosidadActual = memoriaLuminosidad[ubicacion] || 500;
446
447                 // Si hay movimiento y la luminosidad es baja, encendemos la luz automáticamente
448                 if(luminosidadActual < 200) {
449                     notificarAlerta(` Está oscuro en ${ubicacion} (${luminosidadActual} lux) y se ha detectado movimiento
450                         ` <strong>¡Te encendiendo la luz!</strong>`);
451
452                     if(estadoActuadores.luz[ubicacion] === EstadoLuz.APAGADA) {
453                         estadoActuadores.luz[ubicacion] = EstadoLuz.ENCENDIDA;
454                         io.sockets.emit('dashboard:update-actuadores', estadoActuadores);
455                     }
456                 } else {
457                     console.log(`[AGENTE] Movimiento en ${ubicacion}, pero hay suficiente luz (${luminosidadActual} lux)
458                 }
459             }
460         }
461
462         // Regla del Videoportero:
463         if(tipo === 'Videovigilancia' && ubicacion === 'Portal') {
464             if(valor === 1) { // Si se detecta movimiento en el portal, enviamos una alerta de que alguien está en la pue
465
466                 // 1. Notificamos la alerta tanto en el dashboard como por Telegram
467                 notificarAlerta(` <strong>[SEGURIDAD]</strong> ¡Alerta de actividad en el Portal! La cámara de seguridad
468
469                 // 2. Enviamos el GIF visual exclusivamente por Telegram
470                 if (miChatId) {
471                     const gifAlerta = 'https://media.giphy.com/media/Pag51BshRuVqg/giphy.gif'; // GIF de alerta de segur
472
473                     telegramBot.sendAnimation(miChatId, gifAlerta, {
474                         caption: ' Esto es lo que ha capturado la cámara de seguridad en tiempo real. '
475                     }).catch(err => {
476                         console.error('[TELEGRAM ERROR] No se pudo enviar el GIF:', err.message);
477                     });
```

```
478             }
479         }
480
481
482     }
483 }
484
485 // ================================================================================
486 //   Cierre limpio de MongoDB al detener el proceso (Ctrl+C).
487 // ================================================================================
488 // Detecta cuando pulsas Ctrl+C en la terminal para apagar el servidor
489 // y se asegura de cerrar la conexión con MongoDB limpiamente antes de matar el proceso.
490 process.on('SIGINT', async () => {
491         //Si la conexión con el servidor de mongoDB está activa, la cierra
492     if(mongoClient) {
493         await mongoClient.close();
494     }
495
496         //Mata el proceso del servidor
497     process.exit(0);
498 });
499
500 // ================================================================================
501 //                   Arrancamos el sistema por completo
502 // ================================================================================
503 startServer();
```

Para entender cómo funciona este servidor más allá de los comentarios de su código, podemos destacar cinco bloques de implementación clave que estructuran su lógica:

- **Definición Estricta de Estados y Gemelo Digital:** Se hace uso de `Object.freeze` para definir los estados de los actuadores (como `ENCENDIDO`, `APAGADO`, `ABIERTA` o `CERRADA`) de forma inmutable. Esto evita errores de escritura en el código. Estos estados se agrupan en el objeto global `estadoActuadores`, que actúa como el *Digital Twin* en memoria. Cada vez que una interfaz se conecta, lo primero que hace el servidor es enviarle este objeto para asegurar que el usuario vea exactamente lo que está pasando en la casa.
- **Servidor HTTP Nativo :** El servidor lee los archivos HTML directamente desde el disco (*sensor.html* y *dashboard.html*) y los sirve según la ruta que pida el navegador, manejando los errores de carga manualmente.
- **Gestión de Eventos Bidireccionales (Socket.IO):** El corazón de la comunicación en tiempo real se divide en dos tipos de eventos bien diferenciados:
  - *Eventos de entrada (`sensor:data`):* Reciben las lecturas, las guardan en la colección `historial` de MongoDB y las reenvían al

instante a los paneles de control mediante un `broadcast` global (`io.sockets.emit`).

- *Eventos de control manual (`ui:toggle-*`):* Escuchan las órdenes directas que el usuario pulsa en el Dashboard (como encender el aire o bajar una persiana), modifican el Gemelo Digital en el servidor y vuelven a avisar a todas las pantallas abiertas para que se actualicen a la vez.

- **El Agente Lógico:** La función `ejecutarAgente` centraliza la inteligencia del sistema evaluando los umbrales críticos. Encontramos tanto reglas puramente reactivas (apagar el aire si la temperatura baja de 17°C) como **reglas complejas con memoria**. Un ejemplo claro es la regla de movimiento, que consulta primero el objeto `memoriaLuminosidad` para comprobar si la última lectura de luz era baja (¡200 lux), evitando así encender bombillas de forma innecesaria durante el día. Otra curiosidad de este agente es que, dependiendo de si la ubicacion de la medida que se pasa como parámetro tiene un actuador para ese sensor o no, se gestiona de una forma u otra. Esto hace que, si nosotros queremos registrar un nuevo sensor(por ej: un sensor de gas en el baño), no nos ponga ningún inconveniente y por tanto, cada vez que se detecte una anomalía se nos notificará en forma de aviso mediante el *Dashboard* y por el bot de Telegram (**DomoBot Angel**). Ahora bien, como en el Gemelo Digital (`estadoActuadores`) no hay definido un actuador para el gas en la ubicación de Baño, no se activará la correspondiente alarma y sólo nos llegará el aviso. **Con esto, se pretende hacer ver que, podemos añadir tantos sensores como queramos y de varios tipos, pero dependiendo de si tienen actuadores o no, se adaptarán las salidas**.

- **DomoBot Angel** (El bot de Telegram): Como ya hemos comentado, DomoBot Angel 12 es el bot que nos avisa sobre lo que está sucediendo en nuestro hogar, por si no podemos acceder en ese momento al *Dashboard*. Es importante saber que **sólo sirve para notificar, NO para actuar**, es decir, el bot nos muestra avisos sobre anomalías detectadas por nuestros sensores, pero no es posible acceder a los actuadores, para eso hay que acceder al Dashboard. A pesar de esto, no deja de ser un añadido atractivo e importante en el proyecto.

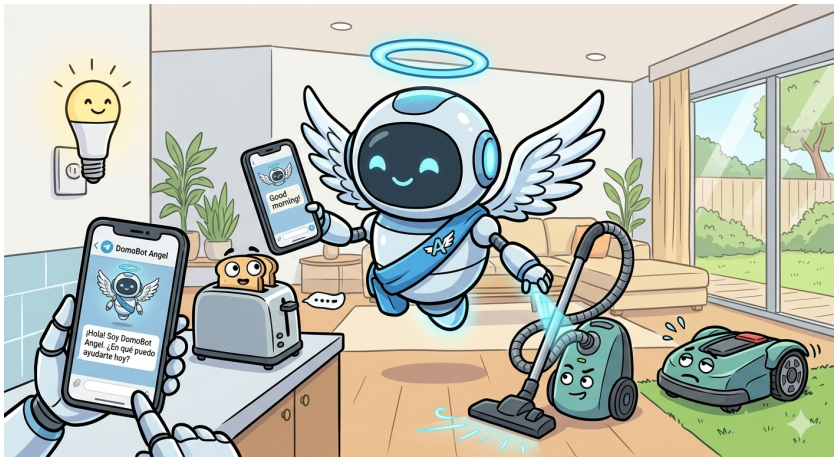


Figura 12: Imagen sobre DomoBot Angel.

3.2.2. Nodos Sensores y Emulación (`sensor.html`)

Los sensores son los .ºjosz .ºídos"de la casa domotizada. Este cliente web está diseñado para emular el comportamiento de un hardware IoT real, y sus tareas principales son:

- **Dar de alta el dispositivo:** Permite al usuario registrar un nuevo sensor indicando su nombre, en qué ubicación de la casa está y de qué tipo es (Luminosidad, Gas, Videovigilancia, etc.).
- **Escoger un dispositivo existente:** Además de poder añadir sensores nuevos, también se puede acceder a los que ya están creados (y almacenados en la colección `sensores` de nuestra base de datos en MongoDB).
- **Envío manual de datos:** Tiene un formulario para poder inyectar valores concretos al servidor y comprobar cómo reacciona el sistema (ideal para hacer pruebas de umbrales críticos).
- **Simulación automática (Modo Piloto Automático):** Es capaz de generar ráfagas continuas de datos aleatorios realistas y enviarlos al servidor de forma constante sin intervención humana. En el código, está fijado a 5000 milisegundos (5 segundos)

Su código es el siguiente:

```
1  <!doctype html>
2  <html lang="es">
3  <head>
```

```
4       <meta charset="utf-8">
5       <meta name="viewport" content="width=device-width, initial-scale=1">
6       <title>Sensor Domótico</title>
7       <style>
8           body {
9               margin: 0;
10              font-family: Arial, Helvetica, sans-serif;
11              background: #f7f7f7;
12              color: #222;
13          }
14
15          main {
16              max-width: 520px;
17              margin: 40px auto;
18              padding: 0 16px;
19          }
20
21          form, .panel-medidas {
22              background: #fff;
23              border: 1px solid #ddd;
24              border-radius: 8px;
25              padding: 16px;
26              display: grid;
27              gap: 12px;
28          }
29
30          label {
31              font-weight: 600;
32              font-size: 0.95rem;
33          }
34
35          input,
36          select,
37          button {
38              padding: 10px;
39              font-size: 1rem;
40              border: 1px solid #ccc;
41              border-radius: 6px;
42          }
43
44          button {
45              border: 0;
46              cursor: pointer;
47              background: #0b5d6b;
48              color: #fff;
49              font-weight: 700;
50              transition: background 0.2s;
51          }
52
53          button:hover {
54              background: #084652;
55          }
56
57          button.secondary {
58              background: #4b5563;
59          }
60          button.secondary:hover {
61              background: #374151;
62          }
63
64          #status {
65              margin-top: 12px;
```

```
            min-height: 1.4rem;
            font-size: 0.95rem;
            font-weight: 600;
        }

        .hidden { /* Sería como la clase OCULTO */
            display: none !important;
        }

        .info-sensor {
            background: #e0f2f1;
            padding: 10px;
            border-radius: 6px;
            font-size: 0.9rem;
            border-left: 4px solid #0b5d6b;
        }
    </style>
</head>
<body>
    <main>
        <h1>Sensor domótico</h1>

        <section id="register-section">
            <div class="panel-medidas" id="panel-existentes" style="margin-bottom: 20px;">
                <h2>Cargar un dispositivo nuevo</h2>
                <label for="select-sensores">¿Deseas usar un sensor ya creado anteriormente?</label>
                <select id="select-sensores">
                    <option value="">-- Selecciona un sensor de la base de datos --</option>
                </select>
                <button type="button" id="btn-cargar-existente">Cargar Sensor Seleccionado</button>
            </div>

            <form id="register-form">
                <h2>Registrar un dispositivo nuevo</h2>

                <label for="name">Nombre del Sensor:</label>
                <input id="name" name="name" type="text" required>

                <label for="location">Ubicación:</label>
                <select id="location" name="location" required>
                    <option value="">Selecciona...</option>
                    <option value="Salon">Salón</option>
                    <option value="Cocina">Cocina</option>
                    <option value="Dormitorio 1">Dormitorio 1</option>
                    <option value="Dormitorio 2">Dormitorio 2</option>
                    <option value="Bano">Baño</option>
                    <option value="Portal">Portal</option>
                </select>

                <label for="type">Tipo de Sensor:</label>
                <select id="type" name="type" required>
                                        <option value="">Selecciona...</option>
                    <option value="Temperatura">Temperatura</option>
                    <option value="Luminosidad">Luminosidad</option>
                    <option value="Gas">Gas</option>
                    <option value="Movimiento">Movimiento</option>
                    <option value="Videovigilancia">Videovigilancia</option>
                </select>

                <button type="submit">Registrar e Iniciar</button>
            </form>

```

```
128             <div id="status"></div>
129         </section>
130
131         <section id="measurement-section" class="hidden"> <!-- Panel de mediciones -->
132             <div class="panel-medidas">
133                 <h2>Panel del Dispositivo</h2>
134                 <div id="sensor-info-card" class="info-sensor"></div>
135
136                 <label for="measurement-value">Introducir Medida Manual:</label>
137                 <input type="number" id="measurement-value" step="0.1">
138
139                 <button id="btn-send-manual">Enviar Medida</button>
140
141                 <hr style="width:100%; border:0; border-top:1px solid #ddd; margin:8px 0;">
142
143                 <!-- Toggle es un botón que funciona como un interruptor -->
144                 <button id="btn-toggle-auto" class="secondary">Iniciar Simulación Automática (Cada 5s)</button>
145             </div>
146         </section>
147     </main>
148
149     <!-- Scripts de JavaScript -->
150
151     <script src="/socket.io/socket.io.js"></script> <!-- Lógica en JS de WebSockets -->
152     <script>
153         const socket = io(); // Conexión al servidor de WebSockets
154
155         // Elementos del DOM
156         const form = document.getElementById('register-form');
157         const status = document.getElementById('status');
158         const registerSection = document.getElementById('register-section');
159         const measurementSection = document.getElementById('measurement-section');
160         const sensorInfoCard = document.getElementById('sensor-info-card');
161
162         const measurementInput = document.getElementById('measurement-value');
163         const btnSendManual = document.getElementById('btn-send-manual');
164         const btnToggleAuto = document.getElementById('btn-toggle-auto');
165         const btnCargarExistente = document.getElementById('btn-cargar-existente');
166         const selectSensores = document.getElementById('select-sensores');
167
168         // Datos del sensor registrado
169         let activeSensor = null; // Indica si el sensor está activo
170         let autoInterval = null; // Para almacenar el intervalo de la simulación automática
171
172         // ============================================================
173         //      CARGAR SENSORES EXISTENTES DESDE LA BASE DE DATOS
174         // ============================================================
175
176         // 1. Nada más conectar, le pedimos al servidor los sensores que ya existen en MongoDB
177         socket.emit('sensor:request-sensors');
178
179         // 2. Escuchamos la respuesta del servidor con la lista de sensores antiguos
180         socket.on('sensor:response-sensors', (lista) => {
181
182             // Limpiamos opciones antiguas excepto la primera
183             selectSensores.innerHTML = '<option value="">-- Selecciona un sensor de la base de datos --</option>';
184
185             // Rellenamos el desplegable con los sensores de la BD
186             lista.forEach(sensor => {
187                 const option = document.createElement('option');
188                 // Guardamos el objeto entero como un string JSON para leerlo luego fácilmente
189                 option.value = JSON.stringify(sensor);
```

```
                option.textContent = `${sensor.name} (${sensor.type} en ${sensor.location})`;
                selectSensores.appendChild(option);
            });
        });

        // 3. Acción al pulsar el botón "Cargar Sensor Seleccionado"
        btnCargarExistente.addEventListener('click', () => {
            if (!selectSensores.value)
                return alert('Por favor, selecciona un sensor válido.');

            // Convertimos el string JSON de nuevo a objeto
            const sensorSeleccionado = JSON.parse(selectSensores.value);

            // Asignamos el ID de Mongo y lo guardamos en activeSensor
            activeSensor = {
                _id: sensorSeleccionado._id,
                name: sensorSeleccionado.name,
                location: sensorSeleccionado.location,
                type: sensorSeleccionado.type
            };

            // Ocultamos el registro y mostramos las medidas
            registerSection.classList.add('hidden');
            measurementSection.classList.remove('hidden');

            // Rellenamos la card de información al igual que si hubiéramos registrado un sensor nuevo.
            sensorInfoCard.innerHTML = `
                <strong>ID en la BD:</strong> ${activeSensor._id}<br>
                <strong>Nombre:</strong> ${activeSensor.name}<br>
                <strong>Tipo:</strong> ${activeSensor.type}<br>
                <strong>Ubicación:</strong> ${activeSensor.location}<br>
            `;
        });


        // ============================================================
        //      FORMULARIO DE REGISTRO DE SENSORES NUEVOS
        // ============================================================
        form.addEventListener('submit', (event) => {
            event.preventDefault(); // Evita que se recargue la página

            // Obtenemos los datos del formulario, creando un objeto JSON
            const payload = {
                name: document.getElementById('name').value,
                location: document.getElementById('location').value,
                type: document.getElementById('type').value
            };

            // Los enviamos al servidor a través del evento 'sensor:register'
            socket.emit('sensor:register', payload);
        });

        // ----- Escuchamos los eventos que el servidor nos envíe -----

        // Escuchamos la confirmacion de registro del sensor
        socket.on('sensor:registered', (sensor) => {
            status.style.color = '#166534';
            status.textContent = `Sensor registrado: ${sensor.name} (${sensor.type}) en ${sensor.location}`;

            // Guardamos los datos del sensor en la memoria del cliente
            activeSensor = sensor;

```

```
252             // Cambiamos la interfaz de usuario para mostrar el panel de medidas
253             form.reset();
254             registerSection.classList.add('hidden'); // Ocultamos el formulario de registro
255             measurementSection.classList.remove('hidden'); // Mostramos el panel de medidas
256 
257             // Mostramos una tarjeta con los datos del sensor registrado
258             sensorInfoCard.innerHTML = `
259                 <strong>ID en la BD:</strong> ${sensor._id}<br>
260                 <strong>Nombre:</strong> ${sensor.name}<br>
261                 <strong>Tipo:</strong> ${sensor.type}<br>
262                 <strong>Ubicación:</strong> ${sensor.location}<br>
263             `;
264 
265             // Ponemos un valor por defecto en la caja segun el tipo de sensor, para facilitar las pruebas manuales
266             switch(sensor.type) {
267                 case 'Temperatura':
268                     measurementInput.value = 22; // ºC
269                     break;
270                 case 'Luminosidad':
271                     measurementInput.value = 300; // Lux
272                     break;
273                 case 'Gas':
274                     measurementInput.value = 0; // ppm (partes por millón)
275                     break;
276                 case 'Movimiento':
277                     measurementInput.value = 0; // 0 o 1 para simular movimiento (0 no, 1 sí)
278                     break;
279                 case 'Videovigilancia':
280                     measurementInput.value = 0; // 0 si no hay nadie, 1 si hay
281                     break;
282                 default:
283                     measurementInput.value = 0;
284             }
285         });
286 
287 
288         // El servidor ha detectado un error al registrar el sensor
289         socket.on('sensor:error', (message) => {
290             status.style.color = '#b91c1c';
291             status.textContent = `Error: ${message}`;
292         });
293 
294         // ============================================================
295         // Envio de medidas al servidor
296         // ============================================================
297 
298         // Enviar la medida introducida al servidor
299         function enviarMedidaAlServidor(value) {
300             // Si no está activo el sensor, no hacemos nada
301             if(activeSensor){
302 
303                 const dataSensor = {
304                     sensorId: activeSensor._id, // Enlazamos la medida con el ID del sensor registrado
305                     name: activeSensor.name,
306                     type: activeSensor.type,
307                     location: activeSensor.location,
308                     value: parseFloat(value)
309                 };
310 
311                 // Emitimos el evento al servidor
312                 socket.emit('sensor:data', dataSensor);
313 
```

```
                // Mostramos un mensaje de confirmación en el cliente
                // Efecto visual temporal en el status para saber que se ha enviado
                status.style.color = '#1e3a8a';
                status.textContent = `[${new Date().toLocaleTimeString()}] Medida enviada: ${value}`;
            }
        }

        // Envío manual mediante el botón
        btnSendManual.addEventListener('click', () => {
            const valor = measurementInput.value;
            if(valor === '') {
                alert('Por favor, introduce un valor para la medida.');
            } else {
                enviarMedidaAlServidor(valor);
            }
        });


        // Envío automático cada 5 segundos al pulsar el botón de toggle
        btnToggleAuto.addEventListener('click', () => {
            if (autoInterval) { // Si ya estaba funcionando, lo paramos
                clearInterval(autoInterval);
                autoInterval = null;
                btnToggleAuto.textContent = "Iniciar Simulación Automática (Cada 5s)";
                btnToggleAuto.classList.remove('secondary');
            } else {
                // Si estaba parado, lo iniciamos
                btnToggleAuto.textContent = "Detener Simulación Automática";
                btnToggleAuto.classList.add('secondary');

                autoInterval = setInterval(() => {
                    let valorLeido;

                    switch(activeSensor.type) {
                        case 'Temperatura':
                            // Simulamos una lectura de temperatura entre 7 y 35 grados
                            valorLeido = (Math.random() * 28 + 7).toFixed(1);
                            break;
                        case 'Luminosidad':
                            // Simulamos una lectura de luminosidad entre 0 y 800 lux
                            valorLeido = (Math.random() * 800).toFixed(1);
                            break;
                        case 'Gas':
                            // Simulamos nivel de gas (ej. partículas por millón). Lo normal es cerca de 0,
                            // pero pondremos picos aleatorios para que salte la alarma.
                            valorLeido = (Math.random() * 500).toFixed(1);
                            break;
                        case 'Movimiento':
                            // Simulamos detección de movimiento (0 o 1) el 40% de las veces
                            valorLeido = Math.random() < 0.4 ? 1 : 0;
                            break;
                        case 'Videovigilancia':
                            // 20% de probabilidad de captar a alguien pasando
                            valorLeido = Math.random() < 0.2 ? 1 : 0;
                            break;
                        default:
                            break;
                    }

                    measurementInput.value = valorLeido; // Actualizamos el input con el valor simulado
                    enviarMedidaAlServidor(valorLeido); // Enviamos la medida al servidor
                }, 5000);
```

```
376             }
377         });
378     </script>
379 </body>
380 </html>
```

Más allá de la estructura visual en HTML y CSS, la verdadera lógica de este cliente reside en su script de JavaScript. Podemos destacar los siguientes bloques clave en su implementación:

- **Validación y Registro Dinámico:** Antes de enviar nada, el cliente comprueba que los campos obligatorios del formulario estén completos. Si todo es correcto, emite el evento `sensor:register` y el servidor se encarga de guardarlo en MongoDB. Además, al cargar la página, el cliente pide al servidor la lista de sensores que ya existen en la base de datos para mostrarlos en un desplegable, evitando duplicados.
- **Motor de Simulación Automática:** Mediante la función `setInterval`, hemos creado un bucle que se repite cada 5 segundos, como ya hemos comentado. Dentro de este bucle, un bloque `switch` analiza de qué tipo es el sensor seleccionado y genera datos matemáticamente creíbles (por ejemplo, temperaturas con decimales entre 7ºC y 35ºC, o valores binarios 0/1 para el movimiento o la cámara). Esto satura de eventos al servidor para probar su rendimiento real.
- **Desacoplamiento Visual/Lógico:** El envío de datos (`enviarMedidaAlServidor`) está programado en una función independiente. Esto permite que tanto el botón manual como el bucle automático reciclen el mismo código para comunicarse con el servidor, manteniendo el script limpio.

3.2.3. Panel de Control de Usuario (`dashboard.html`)

Es la interfaz visual diseñada para los habitantes de la casa. A diferencia de una página web tradicional que requiere recargarla para ver los cambios, este panel es una aplicación **100 % reactiva y asíncrona**. Sus tareas principales son:

- **Monitorización en tiempo real:** Muestra las últimas lecturas de todos los sensores activos de la casa y el registro de alertas críticas.
- **Control interactivo:** Ofrece botones para que el usuario pueda manejar los actuadores (encender o apagar luces, aires acondicionados y alarmas) manualmente

- **CCTV Multimedia:** Muestra una transmisión simulada de la cámara de seguridad del portal. Cuando hay movimiento, el script machaca la propiedad `src` de la etiqueta de imagen para mostrar la alerta roja, y la restaura a verde cuando cesa la actividad.

  Su código es el siguiente:

```
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Dashboard Domótico Central</title>
    <style>
        body {
            margin: 0;
            font-family: Arial, Helvetica, sans-serif;
            background: #f0f4f8;
            color: #333;
        }
        header {
            background: #0b5d6b;
            color: white;
            padding: 15px 20px;
            text-align: center;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
        }
        main {
            max-width: 900px;
            margin: 30px auto;
            padding: 0 16px;
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 20px;
        }
        @media (max-width: 768px) {
            main { grid-template-columns: 1fr; }
        }
        .card {
            background: white;
            border-radius: 8px;
            padding: 20px;
            box-shadow: 0 2px 8px rgba(0,0,0,0.05);
            border: 1px solid #e2e8f0;
        }
        .full-width {
            grid-column: 1 / -1;
        }
        h2 {
            margin-top: 0;
            color: #0b5d6b;
            border-bottom: 2px solid #e2e8f0;
            padding-bottom: 8px;
        }
        .state-value {
            font-size: 1.5rem;
            font-weight: bold;
            margin: 10px 0;
```

```
        }
        .ON { color: #166534; }
        .OFF { color: #b91c1c; }
        .OPEN { color: #2563eb; }
        .CLOSED { color: #475569; }

        button {
            padding: 10px 16px;
            font-size: 1rem;
            border: 0;
            border-radius: 6px;
            cursor: pointer;
            font-weight: bold;
            background: #0b5d6b;
            color: white;
            transition: background 0.2s;
        }
        button:hover { background: #084652; }

        /* Panel de Alertas */
        #alert-container {
            background: #fef2f2;
            border-left: 4px solid #ef4444;
            padding: 12px;
            max-height: 150px;
            overflow-y: auto;
            font-family: monospace;
            font-size: 0.9rem;
        }
        .alert-item {
            color: #991b1b;
            margin-bottom: 4px;
        }

        /* Grid de sensores en tiempo real */
        .sensor-grid {
            display: grid;
            grid-template-columns: repeat(auto-fill, minmax(140px, 1fr));
            gap: 10px;
        }
        .sensor-box {
            background: #f8fafc;
            border: 1px solid #cbd5e1;
            padding: 10px;
            border-radius: 6px;
            text-align: center;
        }
        .sensor-box .val {
            font-size: 1.3rem;
            font-weight: bold;
            color: #0f172a;
        }

        footer {
            background: #0b5d6b;
            color: white;
            text-align: center;
            padding: 20px;
            margin-top: 40px;
            font-size: 0.9rem;
            line-height: 1.5;
            border-top: 1px solid #cbd5e1;
```

```
        }
        footer strong {
            color: white;
            font-size: 1rem;
        }
    </style>
</head>
<body>

    <header>
        <h1> Sistema Domótico Inteligente</h1>
        <p>Panel de Control Central en Tiempo Real</p>
    </header>

    <main>
        <section class="card full-width">
            <h2> Alertas Críticas del Sistema</h2>
            <div id="alert-container">
                <div style="color: #64748b;">No hay alertas activas en este momento.</div>
            </div>
        </section>

        <section class="card">
            <h2> Sensores (Última Medida)</h2>
            <div class="sensor-grid" id="sensors-realtime">
                <p style="color: #64748b; grid-column: 1/-1;">Esperando telemetría de sensores...</p>
            </div>
        </section>

        <section class="card">
            <h2> Control de Actuadores por Zonas</h2>
            <div id="actuadores-container">
                <p style="color: #64748b;">Esperando actuadores del servidor...</p>
            </div>
        </section>

        <section class="card full-width" style="margin-top: 10px;">
            <h2> Circuito Cerrado de Videovigilancia</h2>
            <div
                style="
                    text-align: center;
                    background: #1e293b;
                    border-radius: 8px;
                    padding: 15px;
                    position: relative;
                    box-shadow: inset 0 2px 8px rgba(0,0,0,0.5);">
                    <div
                        id="camera-status"
                        style="
                            position: absolute;
                            top: 25px;
                            left: 25px;
                            color: #22c55e;
                            font-weight: bold;
                            font-family: monospace;
                            font-size: 1rem;
                            text-shadow: 1px 1px 2px black;">
                         LIVE - PORTAL TRANQUILO
                    </div>
                    <img id="camera-feed"
                        src="https://media.giphy.com/media/H4buZ6PLgwsKjA9hlO/giphy.gif"
                        alt="CCTV Portal"
```

```
176                         style="width: 100%; max-height: 380px; object-fit: cover; border-radius: 4px; border: 2px sol
177             </div>
178         </section>
179
180     </main>
181
182     <footer>
183         <strong>Ángel Rodríguez Faya</strong><br>
184         Desarrollo de Sistemas Distribuidos. DSD-3<br>
185         Grado en Ingeniería Informática. UGR.<br>
186         Curso 2025-2026.
187     </footer>
188
189     <script src="/socket.io/socket.io.js"></script>
190     <script>
191         const socket = io();
192
193         // --- CONSTANTES GLOBALES DE VIDEOVIGILANCIA (URLs limpias) ---
194         const URL_CCV_TRANQUILO = "https://media.giphy.com/media/H4buZ6PLgwsKjA9hlO/giphy.gif";
195         const URL_CCV_ALERTA = "https://media.giphy.com/media/Pag51BshRuVqg/giphy.gif";
196
197         // Elementos del DOM
198         const alertContainer = document.getElementById('alert-container');
199         const sensorsRealtime = document.getElementById('sensors-realtime');
200         // const btnToggleAc = document.getElementById('btn-toggle-ac');
201         // const btnToggleBlind = document.getElementById('btn-toggle-blind');
202
203         // Estructura en memoria para no duplicar cards de sensores
204         const sensoresActivos = {};
205
206         // 1. Escuchar actualizaciones de sensores en tiempo real (Broadcast del servidor)
207         socket.on('dashboard:update-sensor', (medida) => {
208             // Si es la primera medida que llega, limpiamos el texto de espera
209             if (Object.keys(sensoresActivos).length === 0) {
210                 sensorsRealtime.innerHTML = '';
211             }
212
213             // Guardamos o actualizamos la medida en nuestro objeto
214             sensoresActivos[medida.sensorId] = medida;
215
216             // Renderizamos las cards de los sensores actualizadas
217             renderSensors();
218
219             // Control dinámico de la cámara del Portal
220             if (medida.type === 'Videovigilancia' && medida.location === 'Portal') {
221                 const feedImg = document.getElementById('camera-feed');
222                 const statusDiv = document.getElementById('camera-status');
223
224                 if (medida.value === 1) {
225                     // GIF de alerta: Alguien merodeando en el portal
226                     feedImg.src = URL_CCV_ALERTA;
227                     statusDiv.innerHTML = " ALERT - ACTIVIDAD DETECTADA";
228                     statusDiv.style.color = "#ef4444";
229                 } else {
230                     // GIF estático: Portal vacío en blanco y negro
231                     feedImg.src = URL_CCV_TRANQUILO;
232                     statusDiv.innerHTML = " LIVE - PORTAL TRANQUILO";
233                     statusDiv.style.color = "#22c55e";
234                 }
235             }
236         });
237
```

```
238         // 1. Función para renderizar las cards de sensores en el dashboard
239         function renderSensors() {
240             sensorsRealtime.innerHTML = '';
241             for (const id in sensoresActivos) {
242                 const s = sensoresActivos[id];
243                 const tipo = s.type;
244                 let unidad = '';
245 
246                 switch(tipo) {
247                     case 'Temperatura':
248                         unidad = '°C';
249                         break;
250                     case 'Luminosidad':
251                         unidad = ' lux';
252                         break;
253                     case 'Gas':
254                         unidad = ' ppm';
255                         break;
256                     default:
257                         break;
258                 }
259 
260                 sensorsRealtime.innerHTML += `
261                     <div class="sensor-box">
262                         <div style="font-size:0.8rem; color:#64748b; font-weight:bold;">${s.name}</div>
263                         <div class="val">${s.value}${unidad}</div>
264                         <div style="font-size:0.75rem; color:#94a3b8;"> ${s.location}</div>
265                     </div>
266                 `;
267             }
268         }
269 
270         // 2. Escuchar cambios de estado en los Actuadores por habitaciones
271         socket.on('dashboard:update-actuadores', (estadoActuadores) => {
272             const container = document.getElementById('actuadores-container');
273             container.innerHTML = ''; // Limpiamos el contenido
274 
275             console.log('Actualización de actuadores recibida:', estadoActuadores);
276 
277             // Extraemos dinámicamente TODAS las ubicaciones de TODOS los actuadores que existan
278             const ubicaciones = [...new Set( Object.values(estadoActuadores).flatMap(actuador => Object.keys(actuador
279 
280             ubicaciones.forEach(ub => {
281                 let tieneActuadores = false;
282 
283                 // Empezamos a mostrar la tarjeta de la ubicación
284                 let htmlZonal = `
285                     <div style="border-bottom: 1px dashed #cbd5e1; padding-bottom: 10px; margin-bottom: 15px;">
286                         <h3> Zona: ${ub}</h3>
287                 `;
288 
289                 // 1. Si esta ubicación tiene Aire Acondicionado lo mostramos
290                 if(estadoActuadores.aireAcondicionado && estadoActuadores.aireAcondicionado[ub] !== undefined) {
291                     tieneActuadores = true;
292                     const acEstado = estadoActuadores.aireAcondicionado[ub];
293                     const acClass = acEstado === 'ENCENDIDO' ? 'ON' : 'OFF';
294 
295                     htmlZonal += `
296                         <div style="margin-bottom: 10px;">
297                             <span
298                                 style="
299                                     font-size:0.9rem;
```

```
                                font-weight: bold;">
                            Aire Acondicionado:
                        </span>
                        <span
                            class="state-value ${acClass}"
                            style="font-size:1.1rem; margin-left:10px;">
                            ${acEstado}
                        </span>
                        <button
                            onclick="socket.emit('ui:toggle-ac', '${ub}')"
                            style="
                                padding: 4px 10px;
                                font-size:0.85rem;
                                margin-left:10px;">
                            Conmutar A/C
                        </button>
                    </div>
                `;
            }

            // 2. Si esta ubicación tiene Persiana lo mostramos
            if(estadoActuadores.persiana && estadoActuadores.persiana[ub] !== undefined) {
                tieneActuadores = true;
                const persianaEstado = estadoActuadores.persiana[ub];
                const persianaClass = persianaEstado === 'ABIERTA' ? 'OPEN' : 'CLOSED';
                htmlZonal += `
                    <div style="margin-bottom: 10px;">
                        <span
                            style="
                                font-size:0.9rem;
                                font-weight: bold;">
                            Persiana:
                        </span>
                        <span
                            class="state-value ${persianaClass}"
                            style="font-size:1.1rem; margin-left:10px;">
                            ${persianaEstado}
                        </span>
                        <button
                            onclick="socket.emit('ui:toggle-blind', '${ub}')"
                            style="
                                padding: 4px 10px;
                                font-size:0.85rem;
                                margin-left:10px;">
                            Conmutar Persiana
                        </button>
                    </div>
                `;
            }

            // 3. Si esta ubicación tiene Alarma de Gas la mostramos
            if(estadoActuadores.alarmaGas && estadoActuadores.alarmaGas[ub] !== undefined) {
                tieneActuadores = true;
                const gasEstado = estadoActuadores.alarmaGas[ub];
                // Usamos la clase OFF (rojo) para ACTIVA (peligro) y ON (verde) para INACTIVA (seguro)
                const gasClass = gasEstado === 'ACTIVA' ? 'OFF' : 'ON';

                htmlZonal += `
                    <div style="margin-bottom: 10px;">
                        <span
                            style="
                                font-size:0.9rem;
```

```
                                font-weight: bold;">
                            Alarma de Gas:
                        </span>
                        <span
                            class="state-value ${gasClass}"
                            style="
                                font-size:1.1rem;
                                margin-left:10px;">
                            ${gasEstado}
                        </span>
                        <button
                            onclick="socket.emit('ui:toggle-gas-alarm', '${ub}')"
                            style="
                                padding: 4px 10px;
                                font-size:0.85rem;
                                margin-left:10px;
                                background-color: #b91c1c;">
                            Conmutar Alarma
                        </button>
                    </div>
                `;
            }

            // 4. Si esta ubicación tiene Luz la mostramos
            if(estadoActuadores.luz && estadoActuadores.luz[ub] !== undefined) {
                tieneActuadores = true;
                const luzEstado = estadoActuadores.luz[ub];
                const luzClass = luzEstado === 'ENCENDIDA' ? 'ON' : 'OFF';

                htmlZonal += `
                    <div style="margin-bottom: 10px;">
                        <span
                            style="
                                font-size:0.9rem;
                                font-weight: bold;">
                            Luz:
                        </span>
                        <span
                            class="state-value ${luzClass}"
                            style="
                                font-size:1.1rem;
                                margin-left:10px;">
                            ${luzEstado}
                        </span>
                        <button
                            onclick="socket.emit('ui:toggle-light', '${ub}')"
                            style="
                                padding: 4px 10px;
                                font-size:0.85rem;
                                margin-left:10px;
                                background: #eab308;
                                color: #000;">
                            Conmutar Luz
                        </button>
                    </div>
                `;
            }

            // Si no tiene ningún actuador, mostramos un mensaje indicándolo
            if (!tieneActuadores) {
                htmlZonal += `<p style="color:#94a3b8; font-size:0.85rem;">Sin actuadores en esta zona.</p>`;
            }
```

```
424
425                 htmlZonal += `</div>`;
426                 container.innerHTML += htmlZonal;
427             });
428         });
429
430         // 3. Escuchar alertas disparadas por el Agente Lógico
431         socket.on('dashboard:alert', (msg) => {
432             // Si es la primera alerta, borramos el mensaje por defecto
433             if (alertContainer.innerHTML.includes('No hay alertas activas')) {
434                 alertContainer.innerHTML = '';
435             }
436
437             const time = new Date().toLocaleTimeString();
438             alertContainer.innerHTML = `<div class="alert-item">[${time}] ${msg}</div>` + alertContainer.innerHTML;
439         });
440
441         // 4. Interacción manual: Pulsar botones para cambiar estados
442         // btnToggleAc.addEventListener('click', () => {
443         //     socket.emit('ui:toggle-ac');
444         // });
445
446         // btnToggleBlind.addEventListener('click', () => {
447         //     socket.emit('ui:toggle-blind');
448         // });
449     </script>
450 </body>
451 </html>
```

Este archivo es el mejor ejemplo de cómo consumir eventos de Socket.io en el Frontend. Los aspectos técnicos más destacables son:

- **Reactividad sin recargas:** El panel es un espectador pasivo que está suscrito constantemente a los eventos globales del servidor (`dashboard:update-sensor` y `dashboard:alert`). Cuando entra un dato nuevo, no recarga la página, sino que utiliza JavaScript para inyectar nuevo código HTML directamente en el DOM, dando una sensación de fluidez instantánea.
- **Generación Dinámica de Actuadores:** Este es uno de los puntos fuertes del diseño. El panel no tiene botones programados (hardcodeados). En su lugar, recibe el Gemelo Digital (`estadoActuadores`) del servidor y utiliza un bucle para extraer qué zonas existen y qué actuadores hay en cada una. Si en el futuro añadimos un ventilador en el servidor, el panel lo dibujará automáticamente sin tener que tocar ni una línea de código HTML.

4. Diagramas de Secuencia

Para comprender de forma visual cómo interactúan los distintos componentes de nuestra aplicación distribuida, se han modelado tres diagramas

de secuencia. Estos diagramas ilustran el ciclo de vida de los datos, desde que nacen en los sensores físicos (o emulados) hasta que el Agente Lógico decide qué hacer con ellos, destacando la naturaleza asíncrona de las comunicaciones.

4.1. Escenario 1: Registro de un Sensor y Telemetría

Este primer flujo representa el escenario base. Cuando un sensor establece conexión mediante WebSockets. El sensor simplemente comienza a emitir sus datos (`sensor:data`). El servidor recibe la carga útil, le añade una marca de tiempo temporal (*timestamp*) y delega la tarea de guardarlo en MongoDB de forma asíncrona mediante una Promesa. De este modo, el hilo principal de Node.js queda libre al instante para seguir atendiendo a otros dispositivos.

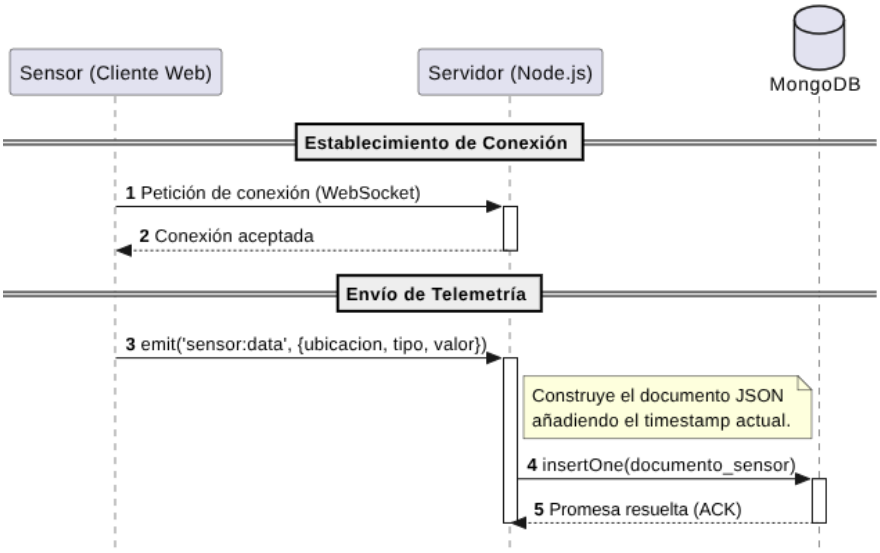


Figura 13: Diagrama de secuencia: Registro y envío de primera telemetría.

4.2. Escenario 2: Evaluación y Envío de un Aviso (Alerta Crítica)

En este escenario, se ilustra la intervención del Motor de Reglas (el Agente Lógico). Al recibir un dato que supera los umbrales de seguridad (por ejemplo, un nivel de gas excesivamente alto), el servidor ejecuta dos acciones concurrentes tras persistir el dato: emite un evento de *broadcast* masivo para que todos los Dashboards conectados parpadeen con la alerta, y

simultáneamente realiza una petición HTTP hacia la API de Telegram para notificar al dueño en su teléfono móvil.

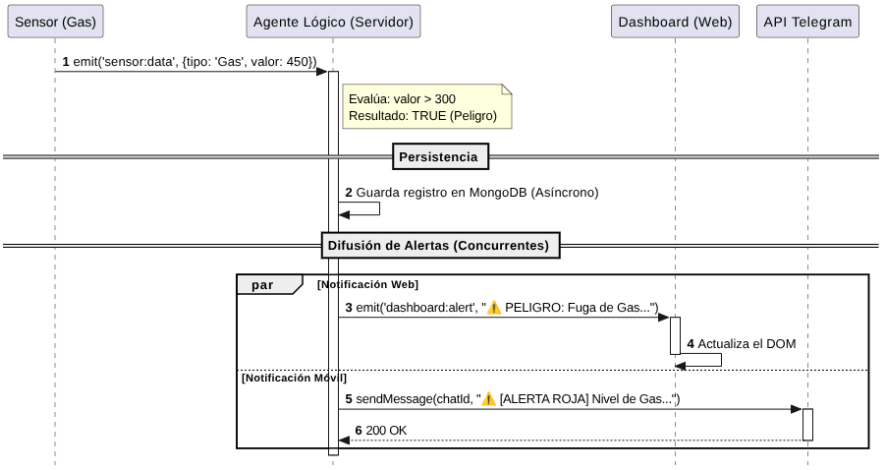


Figura 14: Diagrama de secuencia: Evaluación de reglas lógicas y difusión concurrente de alertas.

4.3. Escenario 3: Envío de Videovigilancia (Bypass Multimedia)

Este es un caso especial dentro de nuestra lógica de control. Cuando la cámara del portal detecta movimiento, el aviso en el Dashboard local es inmediato. Sin embargo, para la alerta externa, el servidor no envía el archivo de vídeo completo desde su propia memoria, sino que hace un *bypass*: le envía a la API de Telegram la URL directa de un CDN de imágenes (Giphy). Es la propia infraestructura de Telegram la que se encarga de descargar el archivo pesado y entregárselo al usuario final, ahorrando así ancho de banda en nuestro servidor domótico.

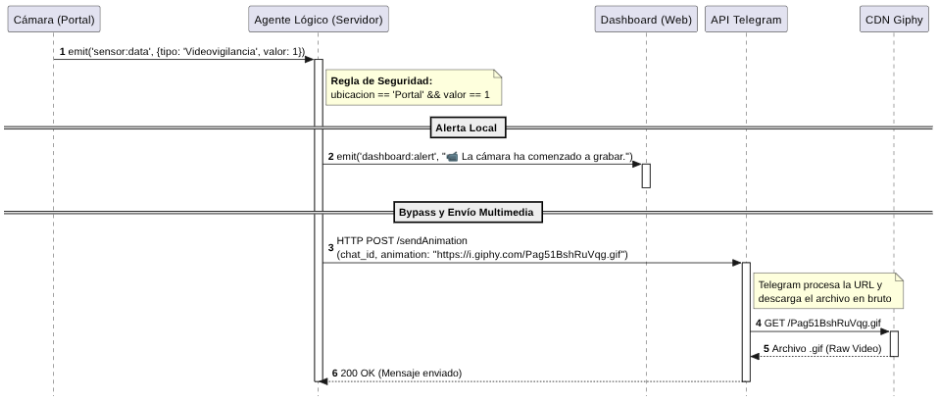


Figura 15: Diagrama de secuencia: Integración de alertas multimedia y consumo de CDN externo.

5. Guía de Uso y Ejemplos de Ejecución

Para poner en marcha el sistema domótico y comprobar su correcto funcionamiento, se ha preparado esta guía práctica paso a paso. El objetivo es detallar desde el encendido inicial de los servicios hasta la simulación de situaciones críticas para ver cómo reacciona nuestro ecosistema distribuido en tiempo real.

5.1. Puesta en Marcha del Sistema

En primer lugar, me gustaría comentar que, la práctica la he desarrollado en mi ordenador personal, que cuenta con las siguientes características:

- **Procesador (CPU):** 11th Gen Intel i5-1135G7.
- **Memoria RAM:** 16 GB DDR4.
- **Sistema Operativo:** Ubuntu 24.04.4 LTS x86_64.

Para arrancar el proyecto, debemos seguir un orden lógico para asegurar que las conexiones entre los componentes (servidor, base de datos y clientes) se establezcan sin errores:

1. **Iniciar la Base de Datos:** Lo primero es comprobar que el servicio de MongoDB está corriendo en nuestra máquina local en el puerto por defecto (`27017`). Esto se hace abriendo una terminal y ejecutando:

```
$ sudo systemctl status mongod
```

Si está corriendo ¡Perfecto! Ya puedes pasar al siguiente apartado. En caso contrario, arrancamos MongoDB con el siguiente comando:

```
$ sudo systemctl start mongod
```

2. **Encender el Servidor Central (Backend):** En una nueva ventana de la terminal, nos situamos en la carpeta del proyecto y lanzamos nuestro script de Node.js. Al hacerlo, la consola nos confirmará que se ha conectado con éxito a MongoDB y que el servidor web ya está escuchando peticiones:

```
$ node registro_sensores.js
```

3. **Vincular DomoBot Angel:** Antes de abrir las webs, buscamos a nuestro bot en la aplicación de Telegram de nuestro móvil u ordenador y pulsamos el botón de **Iniciar** (lo que envía el comando `/start`). Veremos que la consola del servidor detecta el evento y registra nuestro *Chat ID* para el envío de alertas. **Importante:** Tendremos que crearnos nosotros nuestro BOT y poner el token en la constante creada para ello en el Servidor (en `registro_sensores.js`)

5.2. Uso de las Interfaces de Usuario

Con el núcleo del sistema en marcha, abrimos nuestro navegador web e interactuamos con los dos clientes disponibles:

- **Simulador de Sensores:** (Figura 16) Accedemos a la URL `http://localhost:8080/` (o `/sensor.html`). Desde aquí podemos dar de alta un sensor rellenando el formulario o seleccionar uno ya existente del desplegable. Si activamos el "Modo Piloto Automático", el cliente empezará a mandar datos inventados pero realistas cada 5 segundos de forma ininterrumpida.

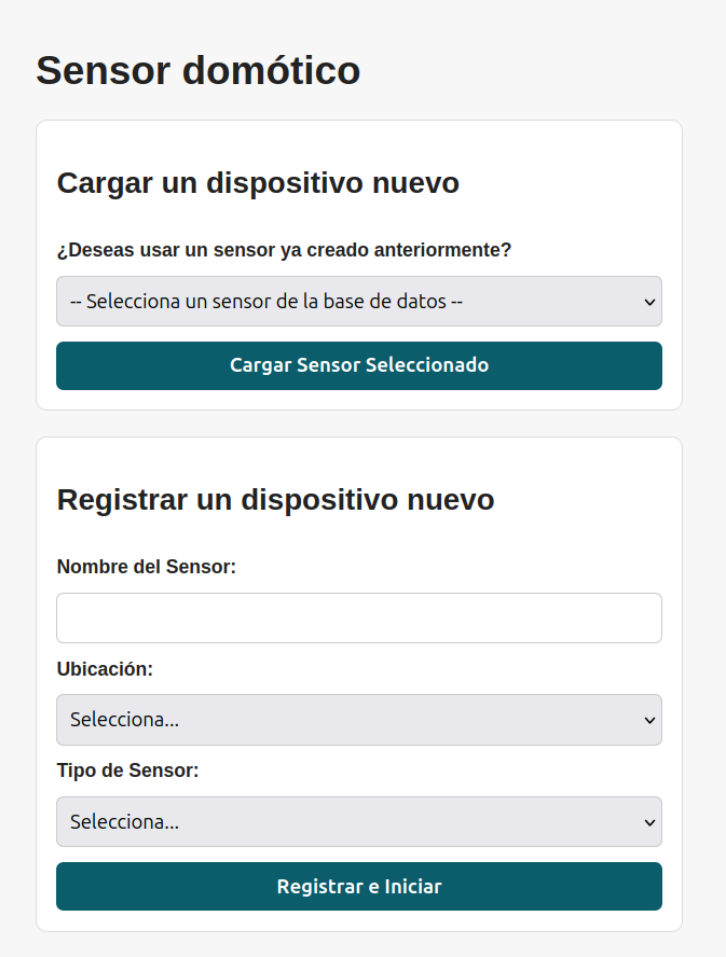


Figura 16: Panel de registro de un sensor `http://localhost:8080/`

- **Panel de Control (Dashboard):** (Figura 17) En otra pestaña del navegador, entramos en `http://localhost:8080/dashboard.html`. Veremos cómo se dibuja la casa dinámicamente gracias al Gemelo Digital y cómo las tarjetas de cada habitación empiezan a actualizarse solas con las medidas que envía el simulador, sin necesidad de refrescar la página.

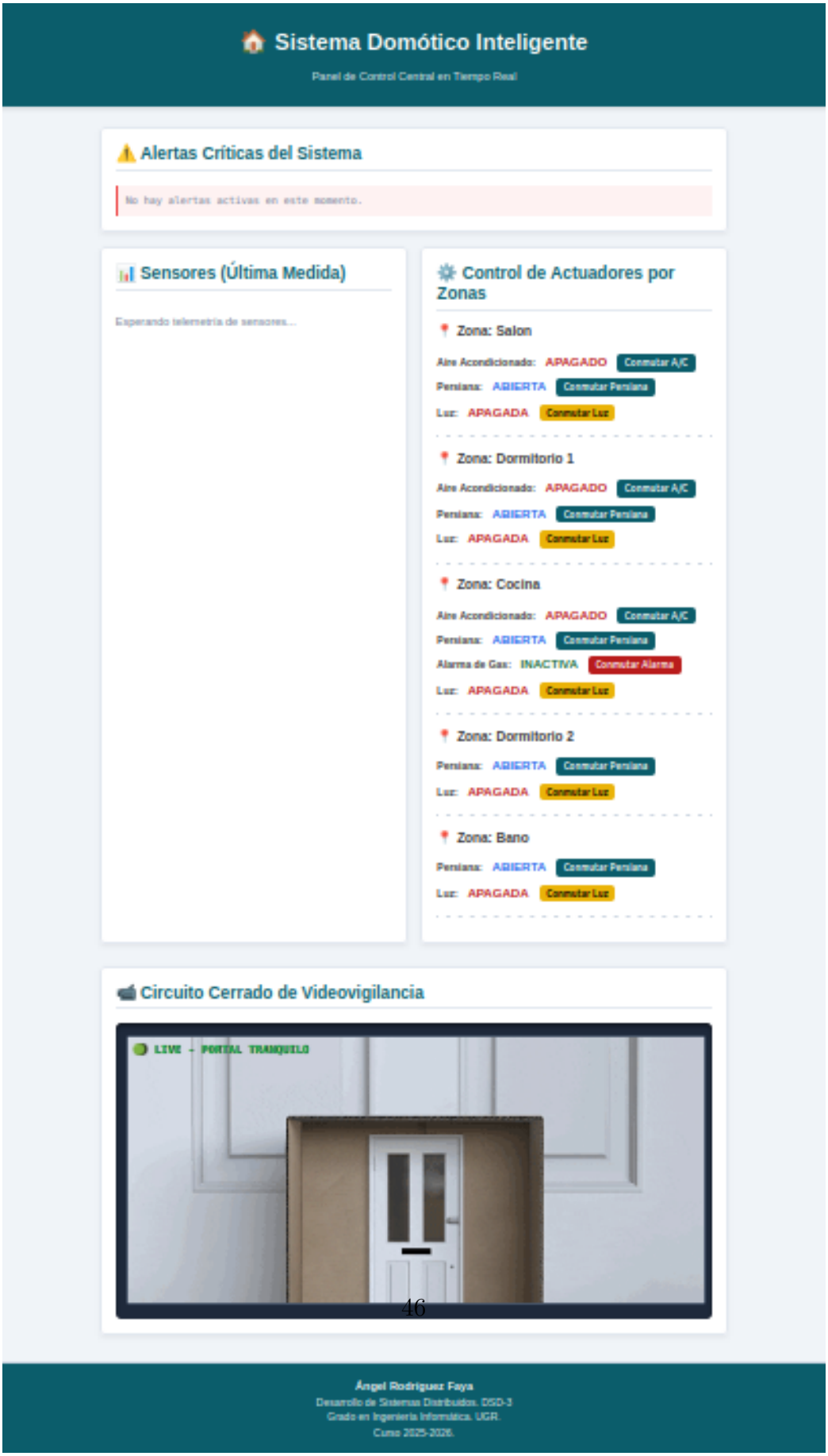


Figura 17: Dashboard de un usuario/habitante de la casa domotizada

- **Iniciar el bot de Telegram**: Le mandamos el comando `/start` (Figura 18)



Figura 18: Inicio de conversación con DomoBot Angel

5.3. Ejemplos de Ejecución y Casos de Prueba

Para demostrar el comportamiento inteligente del Agente Lógico y la perfecta sincronización de los clientes, vamos a ver cada uno de los pan exponen tres casos de prueba reales:

5.3.1. Caso de Prueba 1: Alerta Crítica por Fuga de Gas en la Cocina

En primer lugar registramos el **sensor de gas en la cocina** (Figuras 19 y 20).

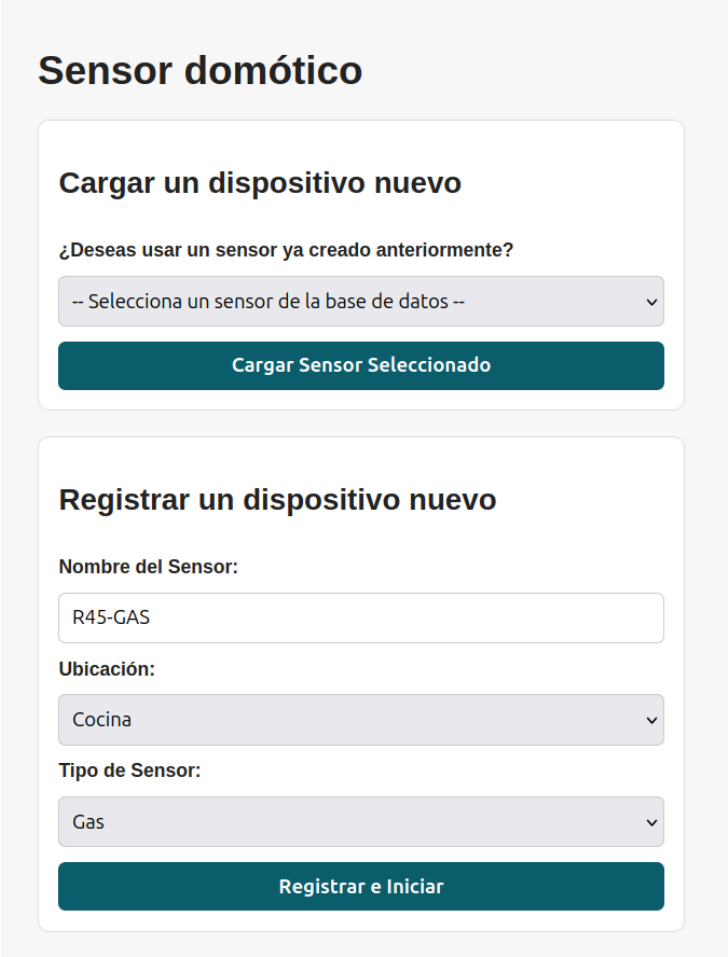


Figura 19: Registramos un sensor de gas para la cocina.

```
ores_Domoticos$ node registro_sensores.js
Servidor basico escuchando en http://localhost:8080
Nuevo nodo conectado: CwqelmqCbS-tUeOyAAAB
Nuevo nodo conectado: ctf-uoSyLCYkETSfAAAD
[TELEGRAM] Usuario registrado para alertas: 5579892961
Sensores registrados actualmente en la base de datos:
```

(index)	name	location	type	registeredAt
0	'R45-GAS'	'Cocina'	'Gas'	2026-05-22T13:21:28.666Z

Figura 20: Salida en la terminal al registrar un sensor de gas para la cocina.

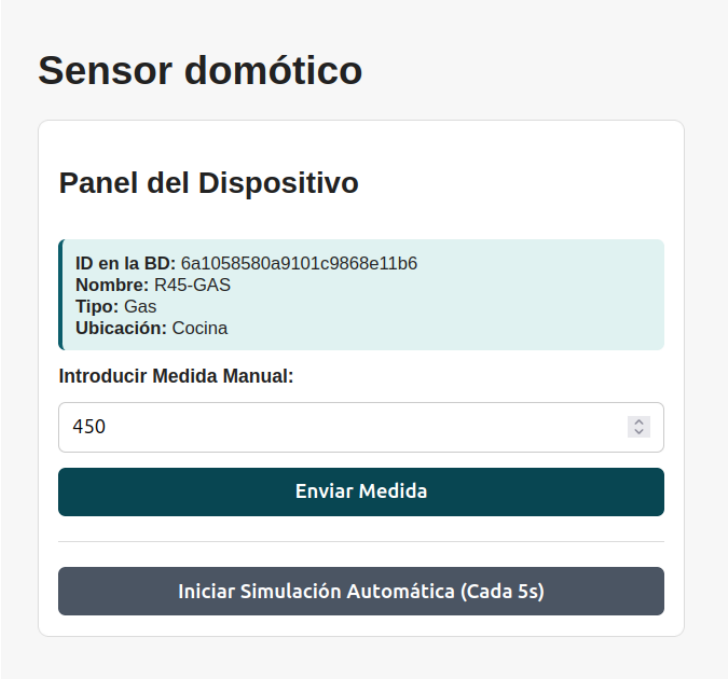


Figura 21: Insertamos una medida de 450ppm para el sensor de gas para la cocina.

```
[BD] Nueva medida registrada: {
  sensorId: '6a1058580a9101c9868e11b6',
  name: 'R45-GAS',
  type: 'Gas',
  location: 'Cocina',
  value: 450,
  timestamp: 2026-05-22T13:21:36.508Z,
  _id: new ObjectId("6a1058600a9101c9868e11b7")
}
```

Figura 22: Salida en la terminal al insertar una medida de 450ppm para el sensor de gas para la cocina.

Si desde la interfaz del sensor inyectamos un valor de gas superior a las 300 ppm (por ejemplo, 450 ppm) (Figuras 21 y 22) en la ubicación de la cocina, ocurre la siguiente reacción en cadena instantánea:

- El servidor persiste la lectura en la colección `historial` de MongoDB. Ya lo hemos visto en la Figura 22

- El *Dashboard* intercepta el aviso (Figura 23), añade una alerta roja en el historial de pantallas y conmuta de forma automática el estado de la Alarma de Gas de la cocina a `ACTIVA`.

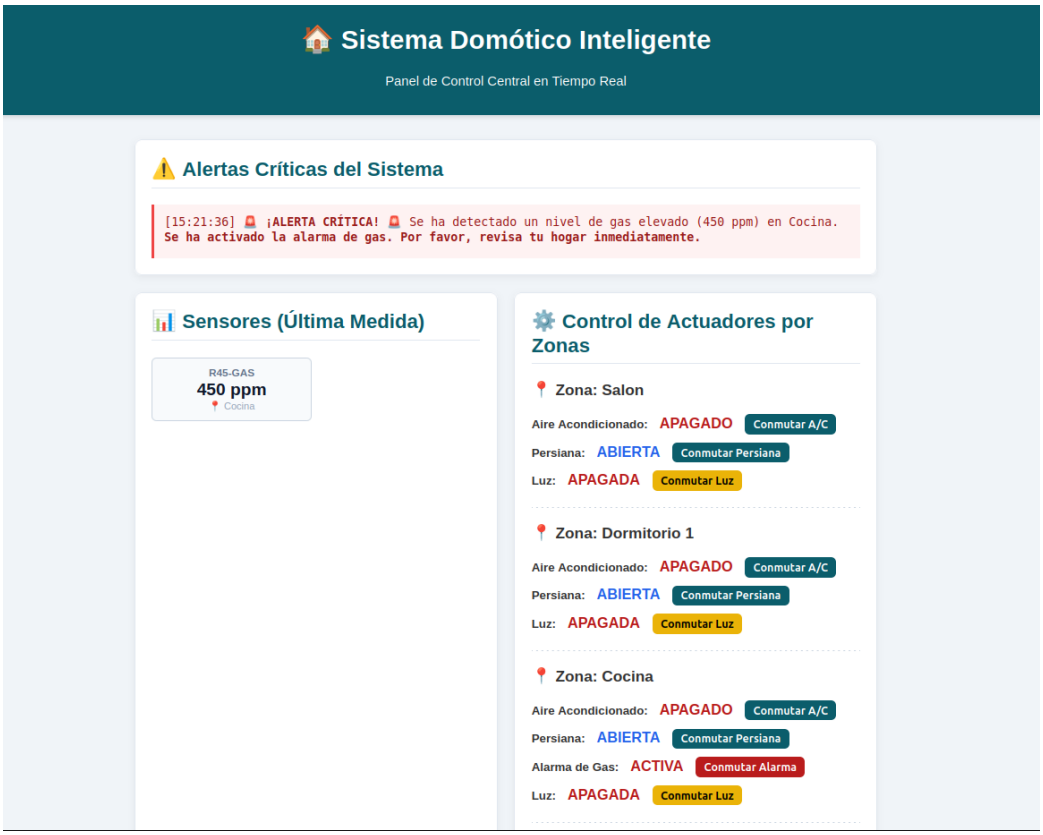


Figura 23: Dashboard muestra el aviso de fuga de gas detectado por el sensor de la cocina

- El usuario recibe una notificación en su teléfono móvil mediante Telegram advirtiéndole del peligro de forma inmediata (Figura 24).

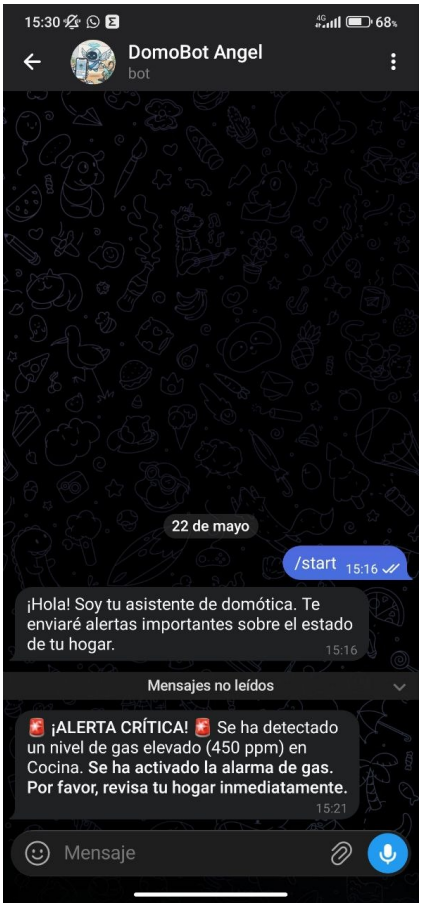


Figura 24: DomoBot Angel nos avisa de la fuga de gas detectado por el sensor de la cocina

5.3.2. Caso de Prueba 2: Regla Compleja Cruzada (Luminosidad y Movimiento)

Para comprobar que el Agente Lógico tiene memoria asociativa y toma decisiones eficientes, realizamos la siguiente prueba:

1. Enviamos desde un sensor una lectura de `Luminosidad` alta (por ejemplo, 600 lux) en el `Salón`. Acto seguido, simulamos presencia (`1`) con un sensor de `Movimiento` en esa misma zona (Figura 25). El servidor registra el movimiento, pero como sabe que hay luz de día, decide no hacer nada. (Figura 26)

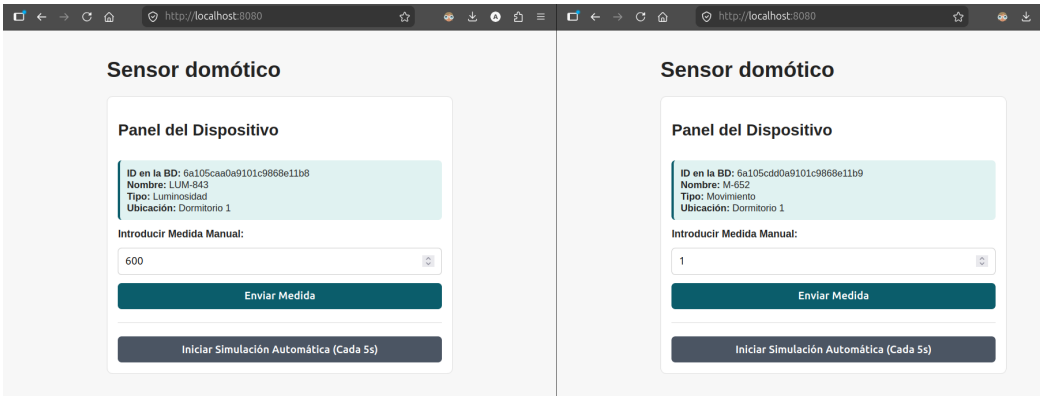


Figura 25: Sensor de Luminosidad y Movimiento en Dormitorio 1 envían su telemetría



Figura 26: Dashboard con nuevas telemetrías

2. Ahora (Figura 27), cambiamos la `Luminosidad` a un valor bajo (por ejemplo, 80 lux) simulando que ha anochecido. Volvemos a mandar un evento de `Movimiento`. Al cruzar ambos datos en memoria, el Agente determina que está oscuro y hay presencia, por lo que cambia el estado de la Luz del Salón a `ENCENDIDA` en el Gemelo Digital, el botón del Dashboard (Figura 28) cambia de color solo y nos llega el aviso. DomoBot Angel también nos avisa (Figura 29)

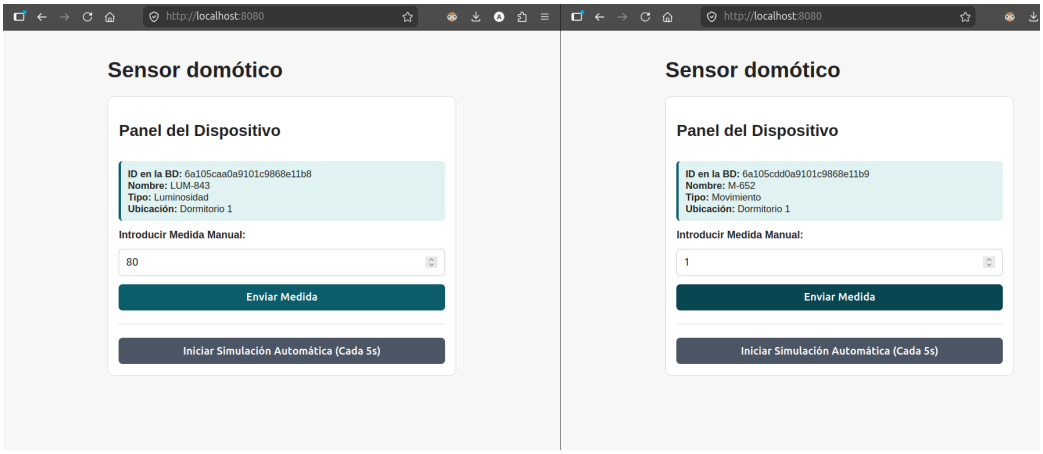


Figura 27: Sensor de Luminosidad y Movimiento en Dormitorio 1 envían su telemetría de nuevo



Figura 28: Dashboard con nuevas telemetrías, nuevo aviso y cambio en el actuador de la Luz del Dormitorio 1.

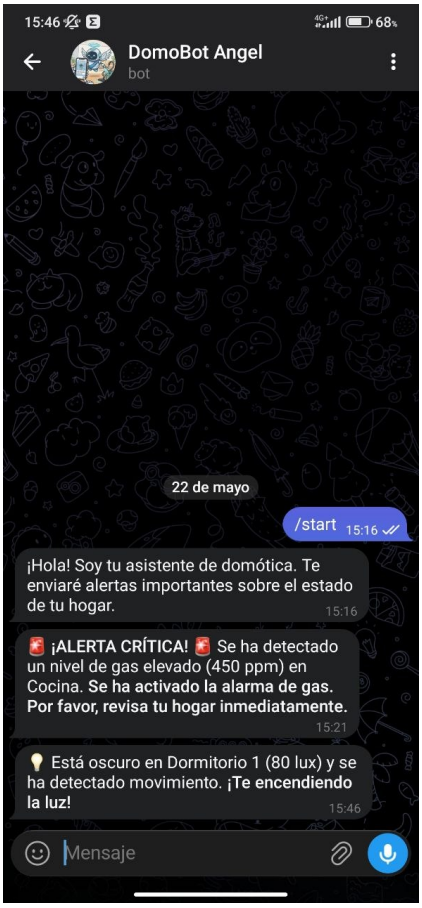


Figura 29: DomoBot Angel nos avisa de que ha enendido la luz debido al evento complejo

5.3.3. Caso de Prueba 3: Seguridad en el Portal (Videovigilancia Activa)

Cuando el sensor de `Videovigilancia` del `Portal` detecta actividad (Figura 30) (un valor igual a `1`), el sistema se pone en modo alerta de seguridad:



Figura 30: El sensor de Videovigilancia ubicado en el Portal detecta movimiento

- El *Dashboard* reacciona (de la Figura 31) cambiando el estilo del circuito cerrado de videovigilancia (CCV) (Figura 32), sustituyendo la imagen estática por una animación interactiva con un marco rojo que simula que la cámara está grabando al intruso.



Figura 31: CCV normal, sin detectar movimiento.



Figura 32: CCV al detectar movimiento.

- El servidor realiza el *bypass* multimedia y le envía al usuario por Telegram un mensaje HTML junto con un GIF animado en tiempo real, permitiendo al dueño ver una simulación visual de lo que ocurre en su puerta desde cualquier parte del mundo. (Figura 33)



Figura 33: DomoBot Angel nos avisa de que hay alguien en el portal y nos envia un pequeño clip de lo que graban las cámaras